

Proof-of-Instant-Settlement: Cryptographically Binding PSP-Attested Instant Payments to Confidential On-Chain State

Juliano Pereira Sales
Harpo-ZK, Brazil
tzdesing@gmail.com

Marco Túlio Rocha Nascimento
Harpo-ZK, Brazil
marcotulio.nascimento@gmail.com

September 30, 2026

Abstract

Confidential-transfer systems on public EVM chains (Zeto, Railgun, Rayls/Enygma) prove that private balance updates are well-formed, but, to the best of our knowledge, none of them binds its private state to a settlement that took place on a real payment rail. We present Proof-of-Instant-Settlement, a construction that binds a specific instant payment, attested by the regulated payment service provider (PSP) that observed it, to a confidential on-chain commitment. A public-EVM contract can then verify that the fiat leg settled for exactly this operation, while the amount, the counterparties and the payment identifiers never appear on-chain. The construction is two-phase: a commitment c_0 to the agreed terms is registered before any payment exists, which gives an asset escrow something to lock onto, and a Groth16 proof later shows that the settlement commitment c carries the same amount and charge identifier together with the rail-level payment identifier, exposing a keyed, single-use settlement tag and a counterparty commitment. An attestation oracle accepts an EIP-712 receipt in which the PSP signs over c_0 , recomputed from the amount it actually settled, rather than over the amount itself, so value consistency is verified without the value being disclosed. We state the security properties with reductions to Groth16 knowledge soundness, Poseidon and Keccak collision resistance and ECDSA unforgeability, and we state the residual institutional assumption, a trusted PSP, explicitly; removing it is the main open problem. The instantiation targets Pix, Brazil’s central-bank instant-payment system, and is evaluated on a public EVM testnet: the settlement circuit has 1,251 R1CS constraints and a complete settlement transaction costs 353,062 gas. We also report a well-determinedness analysis of the circuits, three confidential compliance predicates over the PSP-attested counterparty commitment, and a preliminary post-quantum (ML-KEM) audit-disclosure channel.

1 Introduction

Confidential-transfer systems on EVM chains prove, in zero knowledge, that a private balance update is well-formed (values conserved, ownership valid, no double-spend). This is by now a commodity: Zeto, Railgun, and the Brazilian central-bank-adjacent Rayls/Enygma all rely on the same primitive family (Groth16 or PLONK over BN254/BLS12-381, Pedersen/Poseidon commitments, UTXO or account models with nullifiers) [4, 5]. None of them addresses the *link to a real payment rail*: the on-chain proof says nothing about whether fiat currency actually moved.

Terminology for readers outside Brazil. The concrete rail throughout this paper is *Pix*, the instant-payment system operated by the Central Bank of Brazil (Banco Central do Brasil, Bacen)

since 2020, which settles transfers between accounts at different institutions in seconds, around the clock, over a central infrastructure called the SPI. Participants connect to it through regulated *payment service providers* (PSPs): banks and licensed payment institutions that hold the end-user accounts. A Pix payment is identified by two strings that this paper relies on. The *EndToEndId* (E2EID) is a unique identifier assigned when the payment is initiated and carried end-to-end through the rail; the receiving PSP learns it when the funds are credited, and the SPI enforces its uniqueness. The *txid* is a merchant-side identifier that a payee embeds in a payment request (a QR code or “copy-and-paste” string) before any payment exists, so that the eventual payment can be matched back to the charge that solicited it. Nothing else about Pix is needed to follow the construction; the reader may substitute any instant-payment rail that exposes a unique per-payment identifier and a pre-payment charge identifier, together with a regulated intermediary able to attest settlement.

Relation to prior systems. Proving that a fiat payment happened in order to release an on-chain escrow is *not* new. ZKP2P [6] has done so in production since 2023, using zk-email and zkTLS over consumer payment rails, and is the closest prior art for the general idea. Canton [7] is the incumbent for institutional delivery-versus-payment with privacy, on a permissioned substrate. Our claim is narrower: *within the confidential-transfer family on public EVM chains*, we bind the confidential on-chain state to a specific instant payment as attested by a regulated PSP, and we answer regulatory predicates confidentially. The binding reaches the PSP’s attestation, not the central bank’s own settlement record, which the construction never touches (§8). Relative to ZKP2P, the difference is the institutional setting (a regulated PSP attestation rather than a consumer TLS or email proof) and the confidential-compliance model; relative to Canton and to Brazil’s Drex pilots, it is that this runs on a public EVM chain with no permissioned substrate.

For a settlement use case, such as delivery-versus-payment for a tokenized credit operation, the binding to fiat is the whole point. The paying institution must demonstrate that *a specific instant payment* was made, tied to *this* on-chain operation, while keeping the amount, the parties and the payment identifiers confidential.

What is proved, and by whom. The construction does not cryptographically reach the central bank; it reaches a regulated PSP that observed the rail. Figure 1 shows the protocol flow and this trust boundary. Three different things are called “proof” in this design, and they should not be conflated. *Level 1, the ZK proof*: knowledge of (amount, salt₀, salt, e2eld, txid, k , party, salt_p) consistent with the public commitments (§3), unconditional under (A1). *Level 2, the PSP attestation*: a specific registered key signed a specific statement (§4, Property 7); this is authentication, not truthfulness. *Level 3, rail truth*: the central-bank infrastructure itself attests the payment. The construction does *not* provide Level 3 today; §8 discusses four candidate paths toward it, none evaluated here. What we deliver is Level 1 + Level 2, and the gap to Level 3 is the Trusted-PSP assumption (Assumption (I1)), the central assumption of the system.

The general pattern, Pix as its instantiation. Nothing in §3–§4 is specific to Pix beyond the canonicalization profile of Appendix A: an *external settlement binding primitive* takes (a) a rail-level identifier born only at settlement (an E2EID, but equally a FedNow trace number or a SEPA Instant end-to-end reference), (b) an attestor able to evaluate one hash over what it observed (any ISO 20022 PSP, not only a Pix one), and (c) a pre-existing commitment to bind it to. Proof-of-Instant-Settlement is that primitive instantiated for Pix.

Contribution.

1. A two-phase commitment scheme and a Groth16 circuit `settlement_verify_v2` that binds the agreed terms (amount, txid), registered *before* the payment exists, to the settled payment (`e2eld`), exposing a keyed settlement tag ν and the on-chain anchor K of the key that produced it (§3).
2. An attestation oracle whose receipt is signed over the terms commitment c_0 rather than over the value, a mechanism we call *commitment-carried value consistency*: the attester independently recomputes the commitment from the amount it actually observed settling, so the chain verifies value consistency while no value or payment identifier ever appears on-chain. The mechanism is not specific to this rail: any ISO 20022 attester that can evaluate one hash gains the same property. The receipt carries an expiry, and the oracle exposes an explicit reversal path for the rail’s refund window (§4).
3. Three *confidential compliance predicates* (value within a limit, non-membership in an attested sanctions set, and membership in an attested KYC set) evaluated over the *PSP-attested* counterparty commitment, so that, once a consumer checks them against the settled operation, the counterparty they speak about is the one that actually paid (§5).
4. A crypto-agile, on-chain-governed *post-quantum audit channel* (ML-KEM-768, FIPS-203) and a preliminary in-circuit binding of the disclosure ciphertext to the commitment (§6).
5. A concrete, *measured* implementation report: circuit size at a stated optimization level, proving time, and the gas of the *complete* calls, not of the verifier in isolation (§9).
6. An explicit threat model that states exactly what is and is *not* protected, including liveness, key compromise, and the residual trust in the PSP (§8).

Scope and limitations. Cryptographically, the construction establishes that the same amount and txid underlie c_0 and c (Theorem 1), that the settlement tag is deterministic in `e2eld` under the registered domain key and single-use within a deployed instance (Theorem 3), and that the tag and counterparty commitment accepted at settlement are the ones a registered PSP attested (Theorem 2). Institutionally, it establishes that a registered PSP attested that it observed this payment settle and credit (Assumption (I1)). It does *not* establish that the central bank’s infrastructure settled the payment (§8 discusses four paths toward that, none evaluated here), that the attesting PSP is honest, economic irreversibility of the settlement (§4.2), unlinkability across operations (§8), or soundness under a production-grade trusted setup (the reference artifacts use a single-contributor ceremony, Assumption (S1)).

2 Preliminaries

Instant payment / E2EID. The `EndToEndId` is the unique identifier that accompanies an instant-payment transaction end to end. Its provenance matters for the design: the E2EID is generated *by the payer’s PSP at initiation* (the institution code, ISPB, embedded in it is the *payer’s*), and it exists for transactions that are subsequently *rejected* as well as for those that settle. The central-bank infrastructure (SPI) validates its uniqueness; it does not mint it. Hence *possession of an E2EID does not imply settlement*: an identifier alone is a claim, not a fact, which is why the attestation oracle of §4 exists.

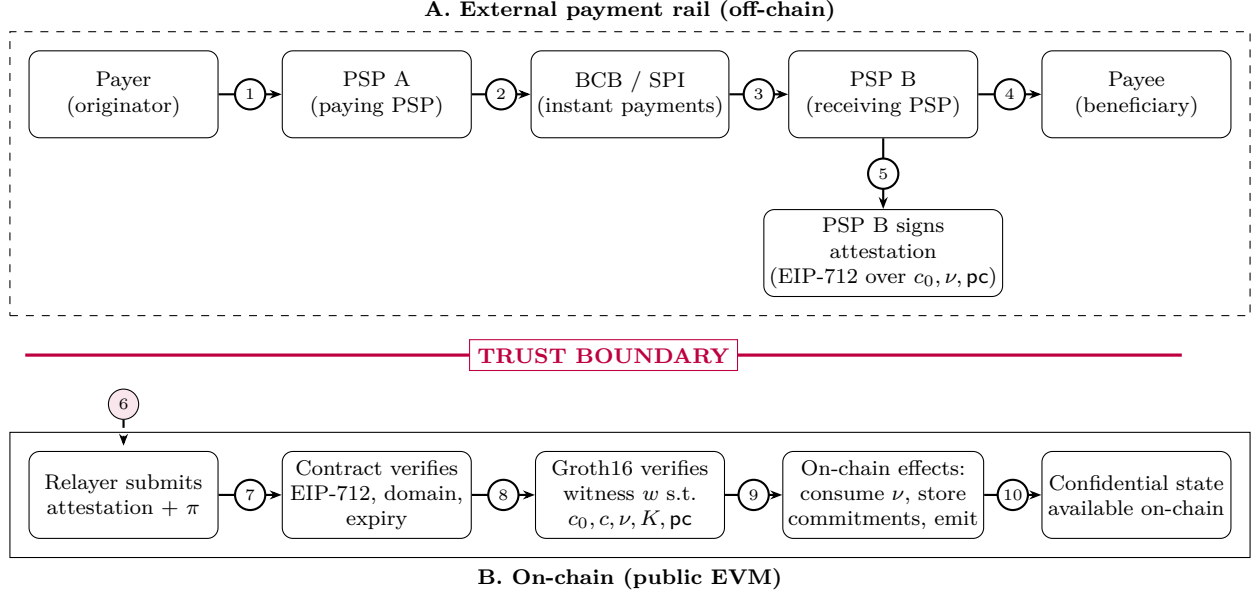


Figure 1: Protocol flow. **A (off-chain)**: (1) the payer initiates a Pix payment; (2)–(4) the rail carries it through both PSPs to the payee; (5) the receiving PSP, the only attestor in the evaluated construction, signs a receipt over the commitments, never over the plaintext value or identifiers. Attestation by the paying PSP as well is future work (path (iii) of §8). **Trust boundary**: everything above is off-chain and PSP-attested; everything below is on-chain and cryptographically checked, and the off-chain payment event reaches the on-chain state only through the PSP attestation and the consistency checks of §3–§4. **B (on-chain)**: (6, 7) a relayer submits the attestation and the Groth16 proof π ; (8) the contract checks the EIP-712 signature, the domain anchor and the expiry; (9) the proof is verified; (10) the tag is consumed and the confidential commitments are recorded. The negative cases of §9 exercise each check in box B individually.

Field encoding and canonicalization. Strings (the E2EID, the PSP charge id `txid`) are mapped to elements of the BN254 scalar field $\mathbb{F}_p$ off-chain via $\text{stringToField}(s) = \text{keccak256}(s) \bmod p$. The hash is taken over the *exact bytes returned by the PSP API*: UTF-8, with no case-folding, no trimming and no re-serialization. The PSP and the institution must hash byte-identical inputs; otherwise the attestation and the proof refer to different field elements and the settlement fails to close. Implementations must therefore treat the API response bytes, not a parsed-and-reprinted string, as the canonical input.

The counterparty identifier committed in `pc` (§3) needs its own canonicalization rule, $\text{CanonicalID}_{\text{rail}}$, and that rule is necessarily *per-rail*. Neither the on-chain contracts nor the circuits parse an identifier format: to them `pc` is an opaque field element, and any national ID or instant-payment rail (a US EIN over FedNow, an IBAN over SEPA Instant, and so on) plugs in by fixing its own $\text{CanonicalID}_{\text{rail}}$ and computing `pc` the same way on both sides. The PSP and the institution must apply the identical rule, since two formattings of the same identifier are two different field elements and `pc` computed by one party would not match `pc` committed by the other. Appendix A gives $\text{CanonicalID}_{\text{Pix}}$, the rule this prototype and its pilot deployment use; it is kept out of the core protocol because the construction is rail-agnostic. Amounts are committed as the integer number of the settlement currency’s smallest unit, parsed exactly from the API’s decimal string, with no rounding and no locale formatting, and the PSP recomputing c_0 must use that same integer: a

value-consistency check that compares two encodings of the same amount fails closed, which is safe but useless.

Poseidon. A ZK-friendly hash over $\mathbb{F}_p$; we use the circomlib instantiations of arity 1 to 4 [3].

Groth16. A pairing-based zk-SNARK with constant-size proofs (8 field elements over BN254, ≈ 256 bytes) and cheap on-chain verification via the EVM `alt_bn128` precompiles (EIP-196/197) [1]. Groth16 requires a per-circuit trusted setup; see §8 for our stance on it.

3 Construction

3.1 Two-phase commitment

A single commitment $c = \text{Poseidon}(\text{amount}, \text{salt}, \text{e2eld}, \text{txid})$, registered once at settlement, would be the simplest design, but it is only computable *after* the payment, because the E2EID is born there. At the moment the parties agree on terms there would be nothing on-chain for an asset escrow to lock onto, so the asset leg could not be conditioned on the payment leg, which is exactly what a delivery-versus-payment composition needs. We therefore split the commitment in two.

At **charge creation**, before any payment exists, the orchestrator registers

$$c_0 = \text{Poseidon}(\text{amount}, \text{salt}_0, \text{txid}), \quad (1)$$

the commitment to the *agreed terms*. The escrow locks on c_0 . At **settlement**, once the payment has happened, the orchestrator registers

$$c = \text{Poseidon}(\text{amount}, \text{salt}, \text{e2eld}, \text{txid}). \quad (2)$$

The circuit proves that the *same* amount and the *same* `txid` appear in both, which ties the settled payment to the terms agreed before it existed. Nothing in the clear goes on-chain at either phase. The blinding factors salt_0 and salt_p , and only these two (never `salt`), are shared with the PSP at charge creation, so that the PSP can recompute c_0 from the amount it actually settled and `pc` from the debtor identifier in the payment message (§4).

3.2 Circuit `settlement_verify_v2`

- **Public signals, in this fixed order:** c_0 , c , the settlement tag ν , the domain-key anchor K , and the counterparty commitment `pc`, i.e. `pubSignals[0..4]`. `SettlementV2.finalize` indexes this array positionally (§3). The order is fixed by the circuit’s public-signal declaration and was cross-checked by hand against the contract’s indexing, because `verifyProof` only checks that the proof is valid *for the signals as given*; what each position means is the calling contract’s responsibility, not the verifier’s.
- **Private signals:** `amount`, salt_0 , `salt`, `e2eld`, `txid`, the domain key k , the counterparty *party*, and salt_p .

- **Constraints:**

$$c_0 \stackrel{!}{=} \text{Poseidon}(\text{amount}, \text{salt}_0, \text{txid}) \quad (3)$$

$$c \stackrel{!}{=} \text{Poseidon}(\text{amount}, \text{salt}, \text{e2eld}, \text{txid}) \quad (4)$$

$$\nu \stackrel{!}{=} \text{Poseidon}(\text{e2eld}, k) \quad (5)$$

$$K \stackrel{!}{=} \text{Poseidon}(k) \quad (6)$$

$$\text{pc} \stackrel{!}{=} \text{Poseidon}(\text{party}, \text{salt}_p) \quad (7)$$

Equations (3) and (4) share the private witness *amount* and *txid*, which is the two-phase binding. Binding *txid* into both commitments prevents a valid proof for one (*amount*, *e2eld*) from being reused for a different charge. The settlement circuit places no range constraint on *amount*; well-formedness of the value comes from the PSP recomputing c_0 from the amount it actually settled (§4), and only the compliance predicate of §5 decomposes the amount into bits.

Settlement tag: one symbol, keyed. We use a single symbol ν for the settlement tag. In ZK terminology ν is a *domain-keyed nullifier*: it is deterministic in *e2eld* under a fixed domain key k and recorded as spent on first acceptance by the deployed instance, but, as Property 6 (§7) states precisely, it is not a nullifier over a global namespace. We use “tag” in the prose that follows, matching the reference implementation. The unkeyed variant $\text{Poseidon}(\text{e2eld})$ is not part of the construction; it is retained only as an ablation row in §9, to make the cost and necessity of the keying legible. The tag, together with the on-chain record of each accepted value as spent, enforces *single use*: a settled payment closes at most one operation. ν must be *deterministic* in *e2eld*, since otherwise the same payment could be replayed under a fresh tag, yet it must not leak *e2eld*. The naïve $\text{Poseidon}(\text{e2eld})$ meets determinism but not hiding: the E2EID is structured and low-entropy, $\text{E<ISPB:8><timestamp:minute><random>}$. The cheap attack is not brute force over the random suffix but *linkage by a party that already holds the E2EID legitimately*. The payer’s own bank, which is not a party to the on-chain operation, knows the E2EIDs it issued and can simply hash them and look for a match. We therefore key the tag: $\nu = \text{Poseidon}(\text{e2eld}, k)$ with k a domain secret carrying ≈ 254 bits.

Anchoring the domain key. A keyed tag raises the question of who holds k . If k lived only with the participants and the PSP, the PSP would become its custodian, and uniqueness would hold only under the informal premise that everyone in a domain used the same key. We therefore register $K = \text{Poseidon}(k)$ on-chain at domain setup and expose it as a public signal of the circuit. Two tags accepted under the same K provably come from the same k , so a given E2EID has a single tag across the whole domain, and single use is enforced by the spent-tag record of the deployed instance (Property 6) rather than by convention. Cross-domain uniqueness is a different claim and rests on the PSP’s $\text{e2eld} \rightarrow \text{txid}$ bijection (Assumption (I1)).

ν is a selective-linkability primitive, not a privacy primitive against the key holders. Anyone who holds k can compute $\text{Poseidon}(\text{e2eld}, k)$ for any *e2eld* it observes and therefore map every ν in the domain back to the E2EID that produced it. In this construction k is held by the attesting PSP, which computes ν itself (§4), and by every institution that proves settlements in the domain, because k is a private input of the circuit (Equation (5)). The domain’s on-chain tag history is therefore linkable, by construction, by its members. This is what makes k useful at all,

and the members are parties already trusted with the payment data (Assumption (I1)). What the key achieves is narrower: *public unlinkability for any observer outside the domain*, against the concrete attack above (a party that already holds E2EIDs legitimately hashing them to find a match), provided that party is not itself a member of the domain. Calling this “privacy” without that qualification would overstate it.

Key rotation. The anchor makes K immutable for the lifetime of a domain, and deliberately so: if a domain rotated k to k' , the same `e2eld` would produce a fresh tag $\nu' = \text{Poseidon}(\text{e2eld}, k')$, and the tag registry, which by design cannot see `e2eld`, would accept a second settlement of an already-settled payment. Rotation is therefore expressed as starting a *new* domain (new K , new escrows), and uniqueness across the two domains is not enforced by the circuit: like cross-domain uniqueness in general, it rests on the PSP’s `e2eld`→`txid` bijection (Assumption (I1)). A deployment that anticipates key compromise should treat this as an operational cost of the design. *Domain separation is a security boundary*, not merely a key-management convenience: a spent-tag record (Property 6) can only recognize a repeated E2EID under the same K , so across domains uniqueness rests on Assumption (I1) alone.

Counterparty commitment. The compliance predicates of §5 speak about a counterparty, and for them to mean anything that counterparty must be the one that actually paid. We therefore make `pc` = `Poseidon`(party, salt_{*p*}) a public signal of the settlement circuit **and** a signed field of the PSP receipt (§4). The PSP computes `pc` over the debtor identifier (CPF/CNPJ) exactly as it appears in the payment message, canonicalized as in §2, using the salt_{*p*} shared at charge creation, and `finalize` accepts only if the attested `pc` equals the proof’s public signal. For charges that do not pin a debtor at creation, the PSP computes `pc` at attestation time, when the payment message names the payer. Without both halves the predicates would be decorative, because the prover would pick the party; with them, the settled operation carries the commitment of the counterparty that paid, and §5 explains how a predicate is tied to it.

3.3 Protocol

1. The receiving institution creates a *real* instant-payment charge through a PSP (e.g. a PSP sandbox), obtaining a `txid`. The commitment c_0 to the agreed terms is registered on-chain and the asset escrow locks on it.
2. The charge (the instant-payment payload string and `txid`) is exchanged *encrypted, peer-to-peer and off-chain* between the two institutions, with an optional on-chain anchor of its hash. It is deliberately *not* written encrypted to the chain. A ciphertext over a payment key, a value and a charge identifier, recorded permanently on a public ledger, is a harvest-now-decrypt-later target that the post-quantum channel of §6 protects for auditor disclosure but would not protect here, and it is personal data written to an immutable medium, a poor posture under the Brazilian data-protection law.
3. The paying institution makes the instant payment; the PSP confirms; the E2EID is resolved.
4. The PSP signs an EIP-712 receipt over $(c_0, \nu, \text{pc}, \text{settled}, \text{deadline})$, recomputing c_0 from the amount it settled and `pc` from the debtor identifier in the payment message; the oracle attests it (§4).

5. The proof π is generated over the public signals $(c_0, c, \nu, K, \text{pc})$. `finalize( $\pi$ )` requires the operation to be `LOCKED` and ν unused, checks the domain anchor K , cross-checks the oracle attestation for ν (same c_0 and same `pc`), then verifies π , and only then consumes the tag and moves the operation to `FINALIZED`. The cheap checks run before the pairing, so a mismatched receipt reverts without paying for verification. The operation becomes `IRREVOCABLE` only after the rail’s refund window elapses without a reversal (§4).

4 The Attestation Oracle

The circuit proves *knowledge* of a preimage; on its own it does *not* prove that the E2EID corresponds to a payment actually settled at the SPI. In a naive design this is resolved purely off-chain: the PSP confirms the payment, returns the E2EID, and the paying institution submits it, an unverified claim. As §2 notes, an E2EID exists for rejected payments too, so the claim is not even self-evidencing.

4.1 A receipt that does not carry the value

The oracle makes the trust link explicit and verifiable on-chain. A trusted PSP (the *attestor*) signs, off-chain, an EIP-712 receipt

$$\text{Receipt}(\underbrace{c_0}_{\text{bytes32}}, \underbrace{\nu}_{\text{settlement tag}}, \underbrace{\text{pc}}_{\text{counterparty commitment}}, \underbrace{\text{settled}}_{\text{bool}}, \underbrace{\text{deadline}}_{\text{uint64}}),$$

and any relayer submits it via `attest(receipt, signature)`.

The choice of fields matters, because the naive alternative leaks. Suppose the PSP signed `Receipt(txid,  $\nu$ , amount, settled)` instead. The contract must recompute the EIP-712 digest to recover the signer, so every signed field appears *in the clear in the calldata* of `attest`, which is public and permanent, and a naive implementation would likely also emit the amount in an event for indexing. Since ν is the same tag the settlement circuit exposes, any observer could join the public settlement (c, ν) to that attestation and read off the amount. Such a design would falsify the confidentiality claim of §8 and Property 5, and the amount would not even be used in any on-chain check.

Signing over c_0 avoids the leak and, unlike signing the amount, *adds* a check. The PSP holds salt_0 and `txid` from charge creation and observes the amount it actually settled; it recomputes c_0 from those three values. **This recomputation is normative: the PSP must evaluate the Poseidon itself (for c_0 over the amount it observed settling, for `pc` over the debtor identifier in the payment message, and for the tag $\nu = \text{Poseidon}(\text{e2eld}, k)$ over the E2EID it observed, using the domain key it holds) and must never sign a c_0 , `pc` or ν value received from the institution.** A PSP whose API merely echoes a client-supplied commitment turns the value-consistency check into a tautology, since the signature would attest that the client’s number equals the client’s number. A client-supplied tag is worse: it would let the institution attach an arbitrary tag to a real payment, severing the anti-replay binding between ν and the E2EID, unless the PSP kept its own `e2eld`→attested registry, which is exactly the state this design avoids. An integration audit should test for this explicitly.

A payment for the wrong amount produces a different c_0 ; the receipt then refers to a commitment the escrow is not locked on, and the settlement does not close. The protocol thus verifies *value consistency without the value appearing anywhere on-chain*. The `deadline` bounds the window in which a signed receipt can be used, which contains the damage from a compromised PSP key provided the attestor chooses short deadlines: the contract enforces `block.timestamp <= deadline` and no upper cap, so a receipt signed with a distant deadline stays usable until then. The receipt

also carries `pc`, the counterparty commitment of §3, which the PSP computes over the debtor identifier in the payment message using the `saltp` shared at charge creation. This adds nothing to what an observer sees: `pc` is already a public signal of the settlement proof, it is a Poseidon image under a ≈ 254 -bit salt, and the attestation and the settlement were already linkable through ν . It adds a check: `finalize` accepts only if the attested `pc` equals the proof’s public signal, so the settled operation carries the counterparty commitment the PSP saw paying, not one the prover chose.

The contract recovers the signer with `ECDSA.recover` over the EIP-712 digest and accepts *iff* the signer is a registered attester, `settled = true`, the receipt has not expired, and ν has never been attested (anti-replay). On success it records ($\nu \mapsto \text{signer}, c_0, \text{pc}, \text{timestamp}$) and emits an event that carries *no* amount, no charge identifier and no party identifier. The reference implementation stores `c0` and `pc` as the single word `keccak256(c0||pc)`, which keeps the storage footprint of the attestation unchanged. The PSP is gasless, as it only signs; the relayer pays gas and has no power to forge, since acceptance is gated on the PSP signature.

Sharing the blinding factors has two consequences. Against the PSP itself, `c0` and `pc` are not hiding at all, by design: the check is the PSP recomputing them, and since the PSP separately holds every other preimage it processed (amount, `txid`, `e2eld`, party and the salts), it is a complete linkability oracle for the operations that pass through it. Nothing in this protocol hides the fiat leg from the PSP that services it, which is a scope boundary of the confidentiality claim (§8). Against any third party that obtains `salt0` (or `saltp`), the hiding of `c0` (or `pc`) degrades from the salt’s ≈ 254 bits to the entropy of the remaining preimage. For `pc` that is the CPF/CNPJ space, enumerable outright; for `c0` it is (amount, `txid`), enumerable only if the charge id (26 to 35 alphanumeric characters) leaks through the same channel, which in practice it does. The salts are therefore secrets of the same sensitivity as the payment data they blind, and the off-chain channel of step 2 in §3 must protect them accordingly. Finally, since `attest` is write-once per tag, a receipt the PSP signs over a *wrong* recomputed `c0` (a software bug, not an attack) permanently burns that ν : the payment can no longer close *that* operation, and the `c0` it was locked on falls back on the liveness timeout below (`cancel`, once the `lock` deadline passes) rather than staying stuck.

Liveness timeout. The oracle cannot force a PSP to sign, so the operation state machine carries its own escape. `lock(c0, deadline)` records a deadline in the future, and once it has passed, `cancel(c0)` lets *anyone* move a still-Locked operation to `Expired`, independent of the PSP. `cancel` and `finalize` both require the `Locked` state and each moves the state away from it, so whichever lands first takes effect, and a valid proof submitted after `cancel` reverts. `SettlementV2` holds no asset of its own: `Expired` is the signal a downstream asset escrow reads, through `isExpired(c0)`, to return its asset to the address it recorded at its own lock time, not to whoever called `cancel`.

4.2 What `settled` means, and reversal

Settlement at the SPI is not the end of the story. The rail provides a return mechanism (certain Pix return and fraud-recovery mechanisms may remain effective for up to 90 days), a dedicated fraud-refund mechanism, and the receiving PSP may place a precautionary hold of up to 72 hours. We therefore define, as the meaning of this protocol’s `settled` field rather than a redefinition of the rail’s own terminology, that `settled = true` means *credit available to the receiver*, not merely “cleared at the SPI”. This is what the PSP must observe before it may sign a receipt with `settled = true` (§4).

Accordingly the oracle exposes `attestReversal`, in which the *same* attester signs ($\nu, \text{returned} = \text{true}, \text{deadline}$) within a configured dispute window, and the operation state machine distinguishes `FINALIZED` from `IRREVOCABLE`: the latter is reached only once the window elapses with no reversal.

IRREVOCABLE is a protocol-state name, not an economic-finality claim. It means only that *this contract* will no longer accept a reversal for the operation, which, as the next paragraph quantifies, is not the same as the rail being unable to reverse the underlying payment for the following ≈ 87 days. A delivery-versus-payment flow must gate the irreversible release of the asset on `isIrrevocable`, not on `isAttested`, and price the residual rail-level risk separately. Revoking an attester blocks *new* attestations and does not invalidate past ones; retroactive invalidation would give the registry owner the power to undo completed settlements. Revocation also leaves the revoked key able to sign reversals of its own attestations within the dispute window, because `attestReversal` checks only that the signer is the original attester. A compromised key can therefore still block the release of the operations it attested in that window (§8).

The state machine admits one orthogonal upgrade: a `RailAnchored` flag, settable at any time by a verified rail-level statement (§8, path iv). It strengthens the evidence behind a `FINALIZED` or `IRREVOCABLE` operation and, applied to returns, makes the post-window residual above visible ex post, without ever sitting on the settlement’s critical path.

The dispute window is a deployment parameter, and the reference deployment sets it to 72 hours. That figure mirrors the rail’s precautionary-hold period, but under the credit-available semantics above it corresponds to *no* rail deadline: a hold is resolved before the PSP attests, and the mechanisms that can still reverse a credited payment (the ordinary return, which may remain effective for up to 90 days, and the dedicated fraud-refund mechanism, on its own track) extend far beyond it. The 72-hour window therefore catches early returns only. An operation that reaches `IRREVOCABLE` after 72 hours remains exposed, for the remaining ≈ 87 days, to a rail-level refund that the on-chain state machine will not represent. That residual risk sits with the asset leg, and a deployment accepts it as the price of not locking capital for three months; a deployment that instead requires `IRREVOCABLE` to dominate the refund window must set the window to 90 days and carry the capital cost. The reference artifacts choose 72 hours.

5 Confidential Compliance Predicates

Beyond proving that fiat settled, a regulated deployment must answer questions about a transaction *without opening it*. We extend the commitment scheme with three ZK *policy predicates*, each a small Groth16 circuit whose only public signals are a commitment and a policy anchor; the sensitive data never appears, *not even to the auditor*. We call them policy predicates rather than compliance predicates on purpose: each proves membership or a bound against an on-chain policy anchor set by a named authority (“Policy authority on-chain” below), which is a necessary input to a compliance determination but not one in itself. Amount-within-limit is a policy condition, not a legal category, and the two membership predicates are named to make the same point.

- **Value within limit** (`compliance_range_verify`): proves amount $<$ limit for an opening of the settlement commitment c , via a bounded comparison (`Num2Bits(64)` range-checking both operands, then `LessThan`).
- **Non-membership in the attested sanctions set** (`sanctions_exclusion_verify`): proves that the counterparty, committed as `pc`, is *absent* from a Sparse Merkle Tree of sanctioned parties (circomlib `SMTVerifier` non-membership, `fnc = 1`). We avoid the name “sanctions clearance”: the predicate proves absence from *the specific set the anchored root represents*, at that root’s version and under whatever normalization built it (item (i) below). It is not a legal determination that the counterparty is unsanctioned, which depends on list freshness, normalization and coverage the circuit cannot see.

- **KYC-set membership** (`kyc_inclusion_verify`): the mirror predicate, which proves that the counterparty is *present* in an allow-list SMT of identities the root’s operator has verified (`fnc = 0`). It is likewise not “KYC compliance”: that determination belongs entirely to the authority that admits identities into the root and governs its lifecycle (who verified, when, under what jurisdiction, whether revoked), and the predicate proves set membership, nothing about the verification behind it.

Whose counterparty? A predicate about a party the prover chose is worthless. If the counterparty commitment were an independent input to each predicate circuit, a malicious institution could commit to any clean identity and prove exclusion and inclusion about *that* identity while settling a payment involving someone else. The construction ties the two together in two steps. First, `pc` is a public signal of the settlement circuit (§3) and a signed field of the PSP receipt (§4), computed over the debtor identifier in the payment message, and `finalize` requires the attested value to equal the proof’s public signal; the settled operation therefore carries the counterparty commitment the PSP saw paying. Second, a predicate proof speaks about whatever commitment it receives as public input, so it is bound to the settlement only when the consumer checks that this commitment is the settled operation’s: for the sanctions and KYC predicates, that `keccak256(c0||pc)` equals the oracle’s stored binding for ν ; for the value predicate, that its commitment equals the c that `Settle` recorded for c_0 . The attestor contracts of our prototype, which are not part of the artifact set of §9, verify each predicate against the pinned policy but do not perform this check; it belongs to the consumer that composes them with the settlement. With it, the counterparty the predicates speak about is the counterparty that paid, under Assumption (I1): a PSP that signs a wrong `pc` is the same trusted-PSP residue as a PSP that signs a false `settled`.

Policy authority on-chain. The policy anchor (the `limit`, or the `sanctionsRoot` or `kycRoot`) must be fixed by an on-chain policy authority, not supplied by the prover; otherwise a malicious institution could prove compliance against a favorable self-chosen limit, or exclusion against an empty sanctions tree. The prototype’s attestors pin the official policy (`setPolicyLimit/setKycRoot`, guarded by a `policyAdmin`) and reject any proof whose public policy signal differs from it. In a separate demonstration on the same testnet, not part of the artifact set of §9, a *genuinely valid* Groth16 proof for a self-chosen loose limit was rejected because it did not match the pinned policy.

Open operational questions. Four questions are left to the deployment. (i) *List normalization*: sanctions lists are keyed differently; OFAC matches on names and aliases, the Brazilian CEIS/CNEP registries on CPF/CNPJ. An SMT over identifiers cannot express name-matching semantics, so a deployment must fix a normalization and accept the resulting false-negative profile explicitly. (ii) *Root lifecycle*: who updates `sanctionsRoot`, how often, and whether proofs issued against a superseded root remain valid. Either answer is defensible, but it must be explicit. (iii) *Per-transaction limit is not AML*: the range predicate bounds a single transaction and says nothing about structuring across many; aggregate monitoring is a different problem, which we do not address. (iv) *Whose identity*: we fix three terms rather than use “counterparty” loosely. `payerId` is the debtor account holder exactly as named in the payment message, which is what `pc` commits. The `economicCounterpartyId` is whoever the transaction is economically for; it coincides with `payerId` except when a third party pays on the debtor’s behalf. The `beneficialOwnerId` is a further AML/KYC concept (who ultimately controls or benefits, e.g. behind a shell entity) that this construction does not represent. The predicates of this section therefore always speak about `payerId`. A deployment must decide whether third-party payments are rejected at attestation, so that `payerId = economicCounterpartyId` always

holds, or accepted with `pc` understood as “who actually paid”; the latter is the more defensible question for AML purposes, but it differs from “who is the beneficial owner”, which this construction leaves to the PSP’s own KYC process.

6 Post-Quantum Audit Disclosure: A Preliminary Construction

This section describes an optional, crypto-agile audit-disclosure channel. It is not part of the core security argument (§7–§8): none of the properties or threat-model rows there depend on it, and a deployment can run the settlement construction with only the classical (X25519) channel. We report it as a preliminary, sized construction.

Auditor disclosure has *long-lived* confidentiality: a ciphertext recorded today must resist a future quantum adversary (“harvest now, decrypt later”). The disclosure channel is therefore crypto-agile and selectable per deployment (classical X25519, post-quantum ML-KEM-768 per FIPS-203 [9], or a hybrid of the two), declared on-chain (`auditChannelKind`, default hybrid) and, in the intended deployment, changeable only through consortium governance, by a timelocked proposal rather than an administrator setter. The reference artifacts, being a research prototype, use a single admin key in place of that governance (the same caveat as the registry owner in §8), so the governance property is *designed*, not *demonstrated*. Each auditor publishes its own encryption key, so no administrator can redirect a disclosure to intercept it.

In-circuit binding. To make a disclosure *verifiable by anyone without decrypting*, we prove in zero knowledge that the published ciphertext encrypts the opening of the commitment under a key derived from the KEM shared secret. Following Qurrency [10], which uses the *same* Groth16/circom/Poseidon/BN254 stack, we port its open-source (Apache-2.0) ML-KEM-in-circom implementation and connect `mlkem_encaps` to our commitment. The resulting circuit `audit_binding_pq_verify` compiles at $\approx 1.62\text{M}$ constraints, so the shared secret is *proven* to originate from ML-KEM rather than asserted. A symmetric-only binding (the shared secret as a private input) is validated on-chain in the interim. The trusted-setup ceremony for the full 1.62M-constraint circuit requires production hardware ($\sim 16\text{ GB RAM}$, 2^{21} powers of tau) and is deferred to the pilot.

Status (preliminary). Unlike the settlement circuits, for which we give proving time and on-chain verification gas, the full in-circuit ML-KEM binding has no end-to-end proving-time or on-chain figure yet, pending the production-scale setup. What is established is that it *compiles and is sized* on our exact stack, and that the selectable channel and the symmetric binding work; the full binding is work in progress. Neither the ML-KEM binding circuit nor the symmetric binding is part of the artifact set of §9, and the reference repository does not include the audit channel.

7 Formal Model and Security Properties

We give a syntax for the construction and state its security properties under explicit cryptographic, operational, institutional and governance assumptions. The table below names the kind of each assumption; not every property reduces to a cryptographic assumption alone, and each statement names the assumptions it depends on. Let $\Pi = (\text{Setup}, \text{Commit}, \text{Prove}, \text{Verify}, \text{Attest}, \text{Settle})$ over the BN254 scalar field $\mathbb{F}_p$:

- $\text{SetupZK}(1^\lambda) \rightarrow (\text{crs}, \text{vk})$: the per-circuit Groth16 common reference string and verification key.

- $\text{SetupDomain}(1^\lambda) \rightarrow (k, K)$: a fresh domain key k and its on-chain anchor $K = \text{Poseidon}(k)$, registered at domain creation and immutable thereafter (§3). The two setups are independent: a domain can be created without a new ceremony, and the ceremony says nothing about any domain’s key.
- $\text{Commit}_0(\text{amount}, \text{salt}_0, \text{txid}) \rightarrow c_0$ and $\text{Commit}(\text{amount}, \text{salt}, \text{e2eld}, \text{txid}) \rightarrow c$.
- $\text{Prove}(\text{crs}, w) \rightarrow (\pi, \nu)$ with witness $w = (\text{amount}, \text{salt}_0, \text{salt}, \text{e2eld}, \text{txid}, k, \text{party}, \text{salt}_p)$ and tag $\nu = \text{Poseidon}(\text{e2eld}, k)$.
- $\text{Verify}(\text{vk}, c_0, c, \nu, K, \text{pc}, \pi) \rightarrow \{0, 1\}$.
- **Attest**: a registered PSP signs (EIP-712) a receipt $\text{Receipt}(c_0, \nu, \text{pc}, \text{true}, \text{deadline})$, recomputing c_0 and pc itself (§4).
- **Settle**: accept iff $\text{Verify} = 1$, the receipt signature verifies against a registered attestor, the receipt is unexpired, its c_0 is the one the escrow locked on, its pc equals the proof’s public counterparty commitment, and the tag is unused.

We assume: **(A1)** Groth16 is knowledge-sound and zero-knowledge under its trusted setup [1]; **(A2)** Poseidon is collision-resistant and, on an input with a high-entropy component, hiding (modeled as a random oracle) [3]; **(A3)** ECDSA/EIP-712 is EUF-CMA; **(A4)** within a domain, txid uniquely identifies the agreed payment obligation and is not legitimately reused for two distinct charges (Charge-identifier uniqueness); **(A5)** keccak256 is collision-resistant for the binding hash the attestation oracle computes over (c_0, pc) (§4). (A4) is what lets txid do the anti-cross-charge work Equations (3)–(4) rely on (§3); it is imposed operationally by the charge-creation flow (the orchestrator mints a fresh txid per charge and the PSP’s own charge API rejects a duplicate), not proved by the circuit, which only checks that *the same* witness txid appears in both commitments. (A5) is stated separately from (A2) because the two hash functions are used for different purposes and neither collision-resistance assumption implies the other: Poseidon binds circuit-internal witness values, while keccak256 binds the oracle’s off-circuit (c_0, pc) pair to a single ν (Theorem 2). The assumptions are of different kinds:

Assumption	Nature	Failure mode if false
(A1) Groth16 sound./ZK	cryptographic	forged proofs / leaked witness
(A2) Poseidon coll.-res.	cryptographic	two openings of one commitment
(A2) Poseidon hiding	heuristic (RO model)	preimage recovery, if the model is wrong
(A3) ECDSA/EIP-712 EUF-CMA	cryptographic	forged attestations
(A4) txid uniqueness	operational	cross-charge proof reuse
(A5) Keccak-256 coll.-res.	cryptographic	oracle binding forged (pc substitution)
(I1) Trusted PSP	institutional	false settled accepted
(S1) Trusted setup	cryptographic, contingent on process	forged proofs, if toxic waste leaked
Policy-root correctness	governance	wrong predicate answers accepted

The cryptographic rows are the ones a reduction can be stated against; the operational, institutional and governance rows cannot be discharged by better cryptography, only by better process.

Knowledge soundness in the presence of honest proofs. The adversaries below observe honest proofs on-chain before producing their own. Groth16 proofs can be re-randomized, so Groth16 is not strongly simulation-extractable, but it is weakly simulation-extractable in the algebraic group

model [2]: an adversary that has seen proofs cannot produce an accepting proof for a *new* statement without knowing a witness for it. The result assumes a linear-independence condition on the QAP polynomials of the public inputs, which snarkjs, the tool that generated the deployed keys, meets by adding one constraint per public input at setup. Our properties are about statements, and this is the notion the reductions below use when they invoke (A1). A re-randomized copy of an honest proof carries the same public signals, hence the same tag ν , which **Settle** marks as used on first acceptance. A deployment that needed non-malleability of the proof string itself, for example to bind a proof to its submitter, would need a strongly simulation-extractable variant or the submitter’s address as a public input.

Property 1 (Correctness). *For an honestly generated $(c_0, c, \nu, K, \text{pc}, \pi)$ of a settled payment with a valid, unexpired PSP receipt, **Settle** accepts.*

Operation binding is stated in three parts, because commitment binding, two-phase consistency and settlement binding are distinct guarantees that a single statement would blur together.

Property 2 (Commitment binding, 2A). *No PPT adversary produces two distinct openings of the same c (or of c_0), except with negligible probability.*

Proof sketch. Two distinct openings of one Poseidon image are a collision, contradicting (A2) directly; this holds independently of Groth16 and of any protocol logic above it. $\square$

Property 3 (Two-phase consistency, 2B). *For any accepting proof π for public inputs $(c_0, c, \nu, K, \text{pc})$, the amount and **txid** used to construct c are identical to those used to construct c_0 .*

Theorem 1 (Two-phase consistency). *Assume knowledge soundness of Groth16 (A1) and collision resistance of Poseidon (A2). Except with negligible probability, every accepting proof admits a witness $w = (\text{amount}, \text{salt}_0, \text{salt}, \text{e2eld}, \text{txid}, k, \text{party}, \text{salt}_p)$ satisfying the circuit relation of §3 (Equations 3–7), and in that relation the same witness values amount and **txid** occur in both commitment equations,*

$$c_0 = \text{Poseidon}(\text{amount}, \text{salt}_0, \text{txid}) \quad (\text{Equation 3}),$$

$$c = \text{Poseidon}(\text{amount}, \text{salt}, \text{e2eld}, \text{txid}) \quad (\text{Equation 4}).$$

Consequently, for an escrow created against $c_0 = \text{Poseidon}(\text{amount}^, \text{salt}_0^*, \text{txid}^*)$, an accepting proof cannot carry, in its post-payment commitment c , an amount or **txid** different from $(\text{amount}^*, \text{txid}^*)$, except with negligible probability.*

Proof. Let π be an accepting Groth16 proof. By knowledge soundness (A1), except with negligible probability there is an extracted witness w satisfying the circuit relation. The circuit has a single witness variable *amount*, used in both Equation (3) and Equation (4), and a single witness variable **txid**, likewise used in both, so there are no independent values $\text{amount}_0, \text{amount}_1$ or $\text{txid}_0, \text{txid}_1$ that could be chosen separately for c_0 and c . Thus, for every accepting witness,

$$\text{amount}_{c_0} = \text{amount}_c = \text{amount} \quad \text{and} \quad \text{txid}_{c_0} = \text{txid}_c = \text{txid}.$$

It remains to relate the extracted witness to the terms the escrow was created with. The extracted witness satisfies Equation (3) for the escrow’s c_0 , so if $(\text{amount}, \text{txid}) \neq (\text{amount}^*, \text{txid}^*)$, then $(\text{amount}, \text{salt}_0, \text{txid})$ and $(\text{amount}^*, \text{salt}_0^*, \text{txid}^*)$ are two distinct preimages of c_0 under Poseidon, a collision (Property 2). Therefore

$$\Pr[\text{Bad}_{2B}(\mathcal{A})] \leq \text{Adv}_{\text{Groth16}}^{\text{KS}}(\mathcal{A}) + \text{Adv}_{\text{Poseidon}}^{\text{CR}}(\mathcal{A}), \quad (8)$$

where $\text{Bad}_{2B}(\mathcal{A})$ is the event that $\mathcal{A}$ produces an accepting proof for an escrowed c_0 whose c carries an amount or **txid** different from the escrow’s terms. $\square$

Corollary 1 (Escrow-to-settlement term consistency). *If an escrow is created against c_0 , any subsequently accepted proof for that escrow uses the same amount and txid in its post-payment commitment c , except with negligible probability under (A1)–(A2). The corollary does not by itself establish that either commitment corresponds to a real-world payment; that semantic fact is outside the cryptographic relation and is handled by the PSP attestation (Property 7) and the Trusted-PSP assumption (Assumption (I1)).*

Property 4 (Authenticated settlement binding, 2C). *Let D be a domain with registered anchor $K_D = \text{Poseidon}(k_D)$, where k_D is the domain secret; where a single domain is fixed, we write K, k without the subscript, and K_D, k_D specialize to exactly those. Let an adversary $\mathcal{A}$ interact with the public settlement system and submit arbitrary proofs and receipts to the attestation oracle and `Settle`.*

An accepted settlement is authenticatedly settlement-bound if, except with negligible probability, there exist a registered attestor P ; an EIP-712 receipt $R = (c_0, \nu, \text{pc}, \text{true}, \text{deadline})$; a valid PSP signature $\sigma = \text{Sign}_P(R)$; and a Groth16 witness $w = (\text{amount}, \text{salt}_0, \text{salt}, \text{e2eld}, \text{txid}, k, \text{party}, \text{salt}_p)$ satisfying the circuit constraints of §3 (Equations 3–7), restated here for convenience:

$$\begin{aligned} c_0 &= \text{Poseidon}(\text{amount}, \text{salt}_0, \text{txid}), & c &= \text{Poseidon}(\text{amount}, \text{salt}, \text{e2eld}, \text{txid}), \\ \nu &= \text{Poseidon}(\text{e2eld}, k), & K_D &= \text{Poseidon}(k), \\ \text{pc} &= \text{Poseidon}(\text{party}, \text{salt}_p), \end{aligned}$$

and, moreover, the receipt authenticated by P is bound to exactly the same c_0 , ν , and pc accepted by `Settle`.

*This is an authentication and cryptographic-consistency property. It does **not** assert that the PSP’s statement `settled = true` is truthful, which remains Assumption (I1).*

Theorem 2 (Settlement binding). *Condition on the deterministic correctness of the deployed `Settle` contract with respect to the acceptance conditions specified in §3 (a property of the deployed code, not a cryptographic hypothesis, and hence not a term in the bound below) and on the registered domain anchor K_D being immutable during the lifetime of domain D (§3). Assume further that (1) Groth16 is knowledge-sound for the deployed `settlement_verify_v2` circuit; (2) Poseidon is collision-resistant over the BN254 field; (3) ECDSA/EIP-712 is EUF-CMA secure; and (4) keccak256 is collision-resistant for the oracle’s binding hash; these are exactly (A1)–(A3) and (A5) above.*

Then, for every probabilistic polynomial-time adversary $\mathcal{A}$, the probability that $\mathcal{A}$ causes `Settle` to accept a settlement for which either (a) no registered PSP signed the accepted tuple $(c_0, \nu, \text{pc}, \text{true}, \text{deadline})$, or (b) the accepted ν is not equal to $\text{Poseidon}(\text{e2eld}, k_D)$ for the `e2eld` contained in an accepting witness for the registered domain, is negligible. More precisely,

$$\Pr[\text{Bad}_{SB}(\mathcal{A})] \leq \text{Adv}_{\text{ECDSA}}^{\text{EUF-CMA}}(\mathcal{A}) + \text{Adv}_{\text{Groth16}}^{\text{KS}}(\mathcal{A}) + \text{Adv}_{\text{Poseidon}}^{\text{CR}}(\mathcal{A}) + \text{Adv}_{\text{Keccak}}^{\text{CR}}(\mathcal{A}), \quad (9)$$

where each term on the right is the standard advantage against the named primitive. The bound holds under the deterministic conditioning above; a contract-correctness failure is a software bug outside the reduction, not a probabilistic event it accounts for.

The theorem binds ν to the `e2eld` contained in the accepting ZK witness; it does **not**, by itself, establish that the PSP observed that same `e2eld`. That correspondence is part of the Trusted-PSP assumption (I1).

Proof. Suppose $\mathcal{A}$ causes `Settle` to accept an operation. By the acceptance condition of `Settle`, the submitted Groth16 proof verifies against the public tuple $(c_0, c, \nu, K_D, \text{pc})$. By Groth16

knowledge soundness, except with negligible probability an extractor obtains a witness $w = (\text{amount}, \text{salt}_0, \text{salt}, \text{e2eld}, \text{txid}, k, \text{party}, \text{salt}_p)$ satisfying every circuit constraint. In particular, the witness satisfies $\nu = \text{Poseidon}(\text{e2eld}, k)$ and $K_D = \text{Poseidon}(k)$, which are Equations (5) and (6) of §3.

Because K_D is the immutable anchor registered for domain D , and the domain key was generated as $K_D = \text{Poseidon}(k_D)$, we have $\text{Poseidon}(k) = K_D = \text{Poseidon}(k_D)$. If $k \neq k_D$, these are two distinct preimages of the same Poseidon output, which under Poseidon collision resistance occurs only with negligible probability. Therefore, except with negligible probability, $k = k_D$, and substituting into (5) gives $\nu = \text{Poseidon}(\text{e2eld}, k_D)$: an accepted proof cannot use an arbitrary domain key or an independently chosen settlement tag without violating Groth16 soundness or finding a Poseidon collision.

It remains to establish that the accepted c_0 , ν , and pc are the values authenticated by the PSP. The attestation oracle accepts a receipt only after recovering the signer from the EIP-712 digest and checking that the signer is a registered attestor, that $\text{settled} = \text{true}$, that the receipt is unexpired, and that ν has not previously been attested. The oracle stores the binding associated with ν as $\text{keccak256}(c_0 \parallel \text{pc})$. Settle then requires that the oracle’s stored binding for the submitted ν equal $\text{keccak256}(c_0 \parallel \text{pc})$ for the c_0 locked by the escrow and the pc supplied as the proof’s public signal.

Therefore, if $\mathcal{A}$ presents an accepted settlement for which no registered PSP signed the exact tuple $(c_0, \nu, \text{pc}, \text{true}, \text{deadline})$, then either (i) the EIP-712 signature was forged or altered, contradicting EUF-CMA security (A3); or (ii) the contract or oracle accepted a tuple not authenticated by the registered attestor, which is excluded by the theorem’s conditioning on deterministic contract correctness rather than by a cryptographic reduction. Similarly, if the accepted pc differs from the PSP-attested pc' , the oracle binding check would require

$$\text{keccak256}(c_0 \parallel \text{pc}) = \text{keccak256}(c_0 \parallel \text{pc}') \quad \text{for } \text{pc} \neq \text{pc}',$$

which happens only with negligible probability under (A5).

Consequently, except with negligible probability, every accepted settlement contains $\nu = \text{Poseidon}(\text{e2eld}, k_D)$ for the e2eld in the accepting witness, and the same ν , c_0 , and pc are authenticated by a registered PSP receipt. This proves authenticated settlement binding. $\square$

Corollary 2 (Counterparty binding). *Under the hypotheses of Theorem 2, an accepted settlement cannot substitute an arbitrary counterparty commitment pc' for the commitment pc authenticated by the PSP, except with negligible probability. Therefore, any compliance predicate of §5 evaluated over this pc , and checked against the settled operation as described there, is bound to the counterparty commitment the PSP authenticated for the settlement. This does not establish that the PSP correctly identified the payer; that semantic claim remains part of Assumption (I1).*

Security-boundary note. Theorem 2 does not claim that the PSP’s statement $\text{settled} = \text{true}$ is objectively true at the payment rail. It establishes only that (1) a registered PSP authenticated the statement; (2) the authenticated statement refers to the same c_0 , ν , and pc used by the accepted settlement; and (3) the accepted ν is cryptographically bound to the e2eld in the Groth16 witness under the registered domain key. The implication “PSP signed $\text{settled} = \text{true} \Rightarrow$ payment actually settled” is outside the theorem and remains the Trusted-PSP assumption of the construction (Assumption (I1)).

Properties 2 and 3 are unconditional under (A1)–(A2). Property 4 is not: it routes through Assumption (I1) for what the attestation *means*, even though *that a registered key produced it* is unconditional under (A3), which is the authentication/truthfulness split of Property 7.

Property 5 (Confidentiality of settlement data, Poseidon random-oracle model). *For any PPT adversary observing the public transcript, namely $(c_0, c, \nu, K, \text{pc}, \pi)$ together with the calldata of `attest` and `settle`, including `Receipt(c_0, \nu, \text{pc}, \text{settled}, \text{deadline})` and its valid EIP-712 signature under a registered attester key, the adversary’s advantage in recovering $(\text{amount}, \text{e2eld}, \text{party}, \text{txid})$ is negligible. The property is scoped to on-chain observers without access to the off-chain secrets $(\text{salt}_0, \text{salt}, \text{salt}_p, k)$; the attesting PSP and the other holders of k are outside this scope, since they hold those secrets (§4). It is a hiding claim about the four excluded values, not a claim that the whole transcript, signature included, can be simulated without the attester’s signing key: the adversary is given a genuine transcript, not asked to distinguish it from a simulation. Signature authenticity is a separate matter, stated as Property 7 under (A3). The hiding argument is a random-oracle heuristic for Poseidon (A2), the standard treatment for a permutation-based hash used as a commitment, not a reduction to a standalone hardness assumption.*

Proof sketch. The transcript includes the receipt, its signature and the calldata on purpose: a transcript restricted to (c, ν, π) would miss the leak of any receipt design whose signed fields carry the amount or txid in the clear (§4), since that calldata is as public and permanent as the settlement itself. We bound the contribution of each public element; this suffices because the elements below are exactly the public elements of the transcript, and none is computable from the others without the hidden witness. (A) *ZK privacy of π .* Groth16 zero knowledge (A1) gives a simulator for π from the public signals $(c_0, c, \nu, K, \text{pc})$ alone. (B) *Commitment hiding of c_0, c, pc, ν .* Each is a Poseidon image with a high-entropy blinding component: c_0 and c under ≈ 254 -bit salts (field elements of the BN254 scalar field), pc under salt_p , and ν under the ≈ 254 -bit domain key k . $K = \text{Poseidon}(k)$ commits to the same key and is constant per domain, so it carries no *per-operation* information beyond identifying the domain. The four images are not independent random variables (c_0 and c share *amount* and *txid*, Property 3, and ν and K share k), but each is individually hiding under the random-oracle heuristic, and the transcript gives no feasible recovery strategy for the excluded preimages beyond the excluded side information. (C) *Metadata is out of scope.* The property says nothing about transaction existence, timing, on-chain addresses (submitter, relayer, attester) or cross-operation linkability from a repeated pc or K ; §8 lists which of these are protected. (D) *The signature authenticates and reveals nothing more.* We do not claim that the signature bits are simulatable: an adversary without the attester’s key cannot produce one, which is EUF-CMA (A3) and the claim of Property 7. What this property needs is narrower: the signed message `Receipt(c_0, \nu, \text{pc}, \text{settled}, \text{deadline})` contains no plaintext of $(\text{amount}, \text{e2eld}, \text{party}, \text{txid})$, since every field is covered by (B) or is a public boolean or timestamp, so a valid signature over it gives the adversary nothing beyond what (B) already accounts for. Composing (A)–(D): every source of information in the transcript either carries negligible advantage toward the four excluded values (A, B) or is out of scope (C), and the unforgeability of the signature (D) is a fact about who signed, orthogonal to what the message reveals. $\square$

Property 6 (Instance-scoped settlement uniqueness). *Within a single deployed (oracle, Settle) instance pair anchored at a fixed, immutable key K , and under the PSP $\text{e2eld} \rightarrow \text{txid}$ bijection of Assumption (I1), each settled payment closes at most one operation. The property does not extend across domains (distinct K , including a rotated one), nor across two instance pairs that share the same K , without that same assumption.*

Theorem 3 (Instance-scoped settlement uniqueness). *For a fixed domain anchor K_D and a fixed (oracle, Settle) instance pair, assuming domain-key immutability (§3), collision resistance of Poseidon (A2), single-use enforcement of ν by the oracle and by Settle (§4, §3), and the PSP’s $\text{e2eld} \rightarrow \text{txid}$*

uniqueness assumption (Assumption (I1)), no PPT adversary can cause two accepted settlements corresponding to the same `e2eld`, except with negligible probability.

Proof. The proof follows the tag argument of Theorem 2, specialized to two accepted settlements rather than one.

- (i) The same `e2eld` under the same domain key k_D produces the same tag: $\nu = \text{Poseidon}(\text{e2eld}, k_D)$ is a deterministic function of `e2eld` once k_D is fixed (Equation (5) of §3).
- (ii) K_D forces the same key across both proofs: by the collision argument of Theorem 2, any proof accepted under the public anchor K_D extracts a witness key k with $\text{Poseidon}(k) = K_D$, and except with negligible Poseidon-collision probability, $k = k_D$ for *every* such proof, so two settlements in the same domain cannot use two different domain keys.
- (iii) `Settle` marks ν as used on first acceptance: on the first accepted settlement for this `e2eld`, it records ν as spent in its own storage (`tagUsed`), and the oracle likewise refuses to attest a tag twice (§4). Both records are per deployed instance; no registry of spent tags indexed by K exists in the reference artifacts.
- (iv) A second settlement for the same `e2eld` presents the same ν (by (i)–(ii)) and is rejected, since `Settle` requires an unused tag (iii).

Therefore, except with negligible probability, no two accepted settlements in the same instance correspond to the same `e2eld`. Cross-domain uniqueness (distinct K_D , including a rotated one) is not implied by this argument, because a fresh domain admits a fresh k_D and hence a fresh ν for the same `e2eld`; it rests entirely on the PSP’s `e2eld`→`txid` bijection, Assumption (I1). Nor is uniqueness across two instance pairs deployed under the same K_D : the artifact record itself deploys a fresh oracle and `Settle` per negative case precisely because the oracle refuses to re-attest a tag, and the same ν is accepted again by the fresh pair. A shared registry of spent tags per K_D , consulted by every `Settle` instance of a domain, would lift the property to the domain; it is listed in §11. $\square$

Property 7 (Attestation unforgeability). *`Settle` accepts only if a registered PSP signed an unexpired receipt with `settled = true` for the operation’s c_0 and for the proof’s counterparty commitment `pc`. This property is authentication of the attestation, i.e. that a registered key produced it, not truthfulness of the underlying settlement claim, which is the residual trust of Assumption (I1).*

Proof sketch. Acceptance recovers the signer via `ECDSA.recover` over the EIP-712 digest (domain-separated by `chainId` and the oracle address) and checks registration and expiry; a forgery breaks EUF-CMA (A3). This does *not* constrain a *malicious registered* PSP, which is the residual Trusted-PSP assumption. $\square$

The proof-based properties (Properties 3, 4 and 6) hold only under (A1)’s trusted-setup premise, which the reference artifacts meet only through a single-contributor ceremony (Assumption (S1) of §8). Commitment binding (Property 2) rests on (A2) alone, and attestation unforgeability (Property 7) on (A3) alone. We now enumerate adversaries informally, including those outside the formal statements above.

8 Security Analysis and Threat Model

We enumerate adversaries and state, for each, whether the construction defends against it.

Adversary / attack	Mitigation (or explicit gap)
Paying institution forges an E2EID it never paid	Mitigated by the oracle: acceptance requires a PSP signature over <code>settled=true</code> . An E2EID exists for rejected payments too (§2), so the identifier alone proves nothing.
Replay: reuse one settled instant payment to close two operations	Mitigated within a deployed instance: the tag ν is single-use, enforced both at the oracle (<code>attest</code>) and at settlement, and the anchor K forces one k per domain (Property 6). Across instances or domains, uniqueness rests on Assumption (I1).
Deanonymization: recover <code>e2eld</code> from the public tag	Mitigated for observers outside the domain by keying the tag with a ≈ 254 -bit domain secret. The threat is not only brute force: a party holding the E2EID legitimately (the payer’s bank, not a party to the operation) could otherwise match by hashing. Holders of k can link by design (§3).
Proof reuse across charges	Mitigated: <code>txid</code> is bound inside both c_0 and c , under (A4) that <code>txid</code> is not legitimately reused across charges.
Settled amount differs from the agreed amount	Mitigated: the PSP recomputes c_0 from the amount it settled; a mismatch yields a c_0 the escrow is not locked on.
Confidentiality: learning amount/parties from chain data	Mitigated: every public element (c_0 , c , ν , K , <code>pc</code> , and the receipt) is a Poseidon image or a proof; a receipt carrying the amount in the clear would break this (§4).
Prover picks a clean counterparty for the compliance predicates	Mitigated at settlement: <code>pc</code> is a public signal of the settlement circuit and a signed field of the PSP receipt, computed by the PSP over the payer identifier in the payment message; <code>finalize</code> requires the attested <code>pc</code> to equal the proof’s public signal. A predicate is bound to this <code>pc</code> only when the consumer checks it against the settled operation (§5). A PSP that signs a wrong <code>pc</code> is Assumption (I1).
A receipt is attested with a wrong c_0 or <code>pc</code> , through a PSP software bug <i>or</i> a compromised or malicious PSP grieving on purpose (the action a bug produces also destroys liveness on demand)	Tag burned, operation recoverable via timeout: <code>attest</code> is write-once per ν , so that specific ν cannot be re-attested, but <code>cancel</code> (§4) lets the operation’s c_0 expire once its <code>lock</code> deadline passes, independent of the PSP. This follows from the anti-replay choice: a PSP holding the attester key can always deny liveness on an operation it is asked to attest for, and <code>cancel</code> bounds the damage to one missed deadline rather than a permanently stuck escrow.
Domain key rotation used to replay a settled payment	Mitigated by construction: K is immutable per domain, a rotation starts a new domain, and cross-domain uniqueness rests on Assumption (I1) (§3).
Soundness: a false proof accepted	Mitigated by Groth16 soundness, <i>under</i> the circuit’s trusted setup (see below).
Attestation forgery: non-PSP submits a receipt	Mitigated: <code>ECDSA.recover</code> and the registered-attester check; the EIP-712 domain binds <code>chainId</code> and the oracle address.
Front-running <code>attest</code> on a victim’s ν	Mitigated: a valid receipt requires the PSP signature over that exact struct.

continued on next page

Adversary / attack (continued)	Mitigation (or explicit gap)
Front-running <code>finalize</code> : an attacker copies a victim's valid proof and receipt and submits <code>finalize</code> first, to steal the outcome	Not applicable: <code>finalize</code> is permissionless by design (any relayer may submit it, like <code>attest</code>), which is safe because <code>SettlementV2</code> moves no asset and pays no address. It only moves <i>the operation keyed by the proof's own c_0</i> to <code>Finalized</code> and emits an event. Front-running changes <i>who paid the gas</i> , never <i>who benefits</i> : an attacker who submits first still moves the same c_0 to <code>Finalized</code> , on behalf of the same operation, and gains nothing by being the caller. A downstream asset escrow built on top must preserve this by keying its release logic on the <i>stored</i> beneficiary from its own lock call, never on <code>msg.sender</code> of whichever transaction observes <code>isReleasable</code> ; otherwise the property does not transfer.
Payment is refunded after settlement (return, fraud refund, precautionary hold)	Mitigated within the dispute window , a deployment parameter (72 h in the reference deployment; under the credit-available semantics a precautionary hold is resolved <i>before</i> attestation, so the window catches early returns only): the attester may register <code>attestReversal</code> and the operation is <code>IRREVOCABLE</code> only after the window elapses. A refund landing after the window (the rail allows up to 90 days) is an accepted residual risk of the asset leg (§4.2).
PSP key compromised	Partially mitigated: receipts carry a <code>deadline</code> , bounding the window of abuse if the PSP signs short deadlines (the contract imposes no cap), and the attester can be revoked for future attestations. A compromised key remains able to attest within the window and, even after revocation, to sign reversals of its own attestations within the dispute window, which blocks the release of those operations (§4.2).
Front-running <code>lock</code> on a known c_0 , or locking it with a hostile deadline	Not mitigated in the contract: <code>lock</code> is permissionless and first-writer-wins (<code>state == None</code>), so anyone who learns c_0 before the legitimate party can lock it with an arbitrary deadline and deny the operation until <code>cancel</code> becomes possible. The asset escrow above this state machine must be the one that calls <code>lock</code> , or gate it to an expected caller.
Liveness: the PSP never signs, leaving the operation stuck	Mitigated: <code>lock</code> takes an explicit <code>deadline</code> , and <code>cancel</code> lets <i>anyone</i> move a still- <code>Locked</code> operation to <code>Expired</code> once it passes, independent of the PSP (§4). <code>cancel</code> and <code>finalize</code> both require <code>state == Locked</code> and each moves the state away from <code>Locked</code> , so they are mutually exclusive: whichever transaction lands first takes effect, and a late-arriving valid proof cannot re-open an expired operation. This state machine has no asset of its own to return (§4); a downstream asset escrow consuming <code>isExpired(c_0)</code> is responsible for returning its asset to whichever address <i>it</i> recorded at its own lock time, not to whoever called <code>cancel</code> , which is permissionless by design.

continued on next page

Adversary / attack (continued)	Mitigation (or explicit gap)
Registry owner unilaterally adds a hostile attester	Partially mitigated: the owner is a single key in the reference artifacts. A production deployment requires a multisig with timelock; otherwise the owner is a trust point as strong as the PSP itself.
The attestation reveals <i>which</i> PSP attested	NOT mitigated. The attester address is public, leaking the receiver’s banking relationship at the institution level. Hiding it would require a ring or threshold signature over the attester set.
Malicious / colluding auditor reads everything	Out of scope of this construction: auditor access is governed by a separate threshold (t -of- n), logged disclosure-authorization contract, not part of the artifact set of §9.
Registered PSP signs a receipt for an unsettled instant payment	NOT mitigated in the current artifacts: this is the residual trust assumption. Path (iv) below would convert it from <i>trusted</i> to <i>accountable</i> : a retrospective rail anchor contradicting the receipt is public, attributable evidence of the false attestation.

Trust decomposition. Claim by claim, what establishes each one:

Claim	Established by
Same amount, same txid across c_0/c	ZK (Property 3)
Tag deterministic in e2eld, single-use per deployed instance	ZK + on-chain record (Property 6)
E2EID genuinely bound to this settlement	ZK + PSP attestation (Property 4)
Counterparty commitment is the one that paid	PSP attestation + ZK cross-check (§5)
Settlement (credit availability) actually happened	PSP attestation alone (Assumption (I1))
Central-bank / rail truth	not established (§8, paths i–iv)
Liveness if the PSP never attests	protocol timeout (cancel , §4)
Finality against a later refund	deployment policy choice, not cryptography (§4.2)
Compliance-predicate policy (limit, sanctions/KYC roots)	policy-root authority (§5)

Only the first two rows are unconditional under (A1)–(A4). Every row below them routes through a named institutional or governance party, not through cryptography alone.

Secret-compromise impact. Distinct secrets protect distinct things:

Compromised secret	Impact
PSP attester signing key	Forge attestations within the receipt’s deadline , for any c_0/pc the attacker chooses; revocable for future attestations (§4). Within the dispute window, also reverse the key’s own past attestations, even after revocation, which blocks their release (§4.2).

continued on next page

Compromised secret (continued)	Impact
Domain key k	Deanonimize every ν ever accepted in the domain back to its <code>e2eld</code> , which the holders of k (the PSP and the proving institutions) already can, by design (“ ν is a selective-linkability primitive”, §3); does <i>not</i> by itself forge an attestation or a proof.
<code>salt₀</code> (or <code>salt_p</code>) for one operation	Degrades that operation’s <code>c₀</code> (or <code>pc</code>) hiding to the entropy of (amount,txid) (or the CPF/CNPJ space), both realistically enumerable (§4). Scoped to the leaked operation, not the domain.
<code>salt</code> (settlement-phase blinding)	Same degradation, for <code>c</code> instead of <code>c₀</code> ; also operation-scoped.
Groth16 toxic waste (trusted setup)	Forge <i>proofs</i> for false statements, which invalidates the reductions behind Properties 3, 4 and 6; this is the most severe compromise on this list (Assumption (S1)). Commitment binding (Property 2) and attestation unforgeability (Property 7) do not depend on the setup.
Registry owner key (policy admin)	Add a hostile attestor or rewrite policy roots; change <code>disputeWindow</code> , which <code>isIrrevocable</code> reads live, and thereby alter the finality of tags already attested; mitigated only by a production multisig-with-timelock, not by this prototype’s single key.

Assumption (Trusted PSP (I1)). *The registered attestor(s) sign `settled=true` only for instant payments the PSP observed as settled and credited, and hold an authoritative mapping (`txid` $\leftrightarrow$ `e2eld` $\leftrightarrow$ amount $\leftrightarrow$ debtor) for each charge: the PSP’s own charge-and-payment records, not an assertion the institution hands it. Without this, nothing stops the PSP from correctly observing a settled payment and signing a receipt that pairs it with the wrong `txid`. The mapping is what gives the recomputation of §4 (“the PSP evaluates the Poseidon itself”) its meaning: a PSP that recomputes `c0` over a self-consistent but wrong tuple produces a valid-looking receipt for the wrong charge. The construction reduces trust from “trust whoever submits the transaction” to “trust a specific PSP key, whose receipts expire and whose every attestation is auditable on-chain”, but does not eliminate it.*

Who the attestor is. The party in a position to confirm that the money was credited is the receiver’s PSP, and a false `settled=true` from it harms its own client, who would surrender the asset leg against a payment that never arrived. The incentive is therefore not aligned with the obvious attack, and the realistic residual risk is closer to “compromised PSP” than to “malicious PSP”. That is an argument for the design, not a proof: an institution under pressure, or an attacker holding the key within the receipt deadline, still produces a valid attestation.

Assumption (Trusted setup (S1)). *Groth16 soundness holds provided the per-circuit setup was performed such that the toxic waste was discarded. The reference artifacts for `settlement_verify_v2` were produced by a phase-2 ceremony consisting of one participant contribution followed by a beacon; both are recorded in the `.zkey` transcript and can be re-verified with `snarkjs zkey verify` against the published `.r1cs` and the Hermes `.ptau`. The participant’s entropy was discarded. This is a single-contributor ceremony run by the authors: it makes δ a secret-derived point rather than the group generator, but it offers no guarantee beyond the honesty of that one contributor, and the beacon used is a fixed placeholder rather than a public, verifiably-future randomness source. A production deployment requires a multi-party ceremony with independent participants and a public beacon. This is a stated limitation of the artifacts, not of the construction.*

Path to removing the PSP assumption. None of the four paths below is part of the evaluated construction of §7–§9. We discuss them to show the ceiling on the trust reduction, and state for each what it removes.

(i) *A signature from the SPI/Bacen infrastructure* over (txid, e2eld, settled) is the only option that takes the PSP out of the trusted computing base entirely. As a *synchronous* service, an online signing endpoint on the critical path of every settlement, it is not publicly available today, and requesting one is a heavy institutional ask. The asynchronous variant is a much lighter ask, and we treat it separately as path (iv).

(ii) *zkTLS/DECO* [8] proves what the PSP endpoint *responded* over a TLS session. This removes trust in the attester signing key and in the reporting channel, but it does not make the PSP honest about the status: a PSP that reports a false **settled** over TLS yields a valid zkTLS proof of a false statement, so zkTLS does not remove the trusted PSP.

(iii) *Dual attestation via open banking* is an intermediate step that we leave to future work. Under Open Finance Brasil, responses from the *payer’s* institution are signed (JWS, PS256) and carry the **endToEndId** and status. Combining that with the receiver’s PSP attestation turns “trust a PSP” into “trust that two regulated institutions on opposite sides of the same E2EID are not colluding”, which is still an assumption, but a stronger one.

(iv) *Optimistic attestation with retrospective rail anchoring.* The attestation does not have to be trustless *at settlement time* to stop being merely trusted; it is enough that it becomes verifiable *after the fact*. The receiver-PSP receipt of §4 gives optimistic finality: the operation reaches **FINALIZED**, and **IRREVOCABLE** after the dispute window, as today. A rail-level statement landing minutes or days later then either hardens the operation or contradicts it. The state machine gains one orthogonal flag, **RailAnchored**, settable at any time after attestation. An anchor consistent with the receipt upgrades the operation’s evidence from “a regulated party said so” to “the rail said so”. Under a completeness guarantee for the signed root, stated next, a receipt that cannot be anchored stands contradicted: once the rail statement for that period exists, either the institution presents the membership proof, which hardens the operation, or its absence after a grace period becomes the signal, and any party holding e2eld and k (the attesting PSP, or an auditor under the disclosure of §6) can prove non-settlement outright with the same SMT non-membership predicate the sanctions circuit already uses.

This last step is only as strong as the completeness of the signed root. Absence from a Merkle root proves non-settlement only if the root is published with an explicit claim of the form “this root commits to the complete set of E2EIDs settled in interval T ”, made by the same central-bank infrastructure that signs it. Otherwise a delayed batch, an incomplete window, a synchronization gap or a publisher omission is indistinguishable from genuine non-settlement, and absence proves nothing. This is a separate assumption on top of the signature itself, and path (b) below does not deliver non-membership proofs until that completeness claim exists for the specific rail artifact used. With it, Assumption (I1) stops meaning *trusted* PSP and starts meaning *accountable* PSP: the attester can still lie, but the lie becomes provable and attributable ex post, which is the deterrent regulated institutions answer to. We see two realizations, in increasing order of independence.

(a) *Reuse what the rail already signs.* The SPI’s own settlement messages, the **pacs.002** status report delivered to the payer’s PSP and the settled **pacs.008** forwarded to the receiver’s PSP, are statements of the fact at issue, held today by the PSPs of both legs. Both are cataloged ISO 20022 messages of the SPI, specified in Volume VI (Instant Payments) of the SFN Service Catalog and carried over the RSFN [14]. We do not claim a specific signature algorithm or certificate chain: the catalog documents the message set and transport, but we have not confirmed the exact signature scheme against the Bacen security manual. Verifying that signature inside a circuit (the zk-email

line, with the rail’s key as public anchor) or inside an attested TEE, and exposing only the tag, turns an artifact that already exists into a public anchor. The natural prover is the receiving institution itself, which already holds `e2eld` and k , so no new custodian of the linkage secret is created.

(b) *The policy ask, restated cheaply.* Instead of an online signing service, the central-bank infrastructure could publish a periodically *signed accumulator* of settled E2EIDs, a Merkle root built with the SMT machinery of §5. The institution then proves membership of its `e2eld` under the signed root in zero knowledge, exposing only ν . Asynchrony is what makes (b) plausible: it puts nothing on the settlement’s critical path, batches trivially and tolerates delay, because the optimistic leg already closed the operation.

Two boundaries apply to path (iv). First, the rail anchors *settlement at the SPI*, not *credit availability*, so the receiver PSP’s receipt remains necessary for the settled-means-credit semantics of §4.2; returns (`pacs.004`) traverse the SPI symmetrically and can be anchored the same way, which would also make the refund window of §4.2 visible on-chain ex post. Second, the lesson of §4 applies unchanged: neither the signed rail message nor any statement carrying the E2EID in the clear goes on-chain. Verification happens off-chain or in-circuit, and only the tag and a boolean are anchored.

Path	Removes	Does not remove
This work’s PSP receipt	submitter trust (anyone can relay)	PSP honesty about settled
(ii) zkTLS/DECO	attestor signing key, reporting channel	PSP correctness of the reported status itself
(iii) Dual PSP (open banking)	single-institution trust	collusion between the two
(i) Central-bank signature	PSP trust entirely	nothing (assuming SPI/Bacen key custody), but not a synchronous service today
(iv-a) Signed rail artifact	PSP trust, retrospectively	freshness (evidence lands after finality)
(<code>pacs.002/004</code>)		
(iv-b) Signed accumulator + membership	online signing service (asynchronous)	completeness of the root (stated above)

None of these rows composes with another automatically: (iv-a) and (iv-b) still rest on the SPI/Bacen infrastructure choosing to publish the underlying artifact, which is a policy fact about the rail, not a cryptographic one.

Privacy model: what is and is not hidden. The construction hides, on-chain, the payment *amount*, the *parties*, and the payment identifiers (`e2eld`, `txid`): these values appear only through Poseidon commitments or ZK witnesses, and the attestation receipt does not carry them in plaintext, with negligible leakage under (A2). It does *not* hide (i) the *existence* and *timing* of an operation, since a settlement transaction is public; (ii) the *on-chain addresses* that submit the settlement and relay the attestation, and the *attestor address* itself; or (iii) the fiat leg itself, which the settlement rail (the SPI and the PSPs) observes by construction. Address-level linkability is out of scope and, if required, must be handled by an external anonymity layer. The construction conceals the amount and identifiers *on-chain*, not the fact that a payment traversed the rail. Regulatory access to the underlying data is a separate disclosure channel (§6), outside the on-chain confidentiality claim.

Property	Protected?
Amount confidentiality	Yes
e2eld / txid confidentiality	Yes
Party identifier confidentiality	Yes
Transaction existence and timing	No
Submitting / relaying address privacy	No
Attestor (PSP) address privacy	No
Cross-operation linkability (repeated pc, K, addresses)	No

Confidentiality is not unlinkability, and the construction does not claim the latter. Even an observer who never learns amount, e2eld or party can still correlate operations by the same pc recurring across settlements (a counterparty that transacts more than once is linkable by design, because pc exists so that predicates can be evaluated consistently against the same identity); the same submitting or relaying address; the same attestor address (which operation went through which PSP); the same domain anchor K; and ordinary transaction timing. None of these is mitigated by this construction; an external anonymity layer would be needed for any of them, and none is currently in scope.

9 Implementation and Benchmarks

The circuit is implemented in circom 2.2.3 and the contracts in Solidity 0.8.27; the Groth16 verifier is generated by snarkjs 0.7.6. All figures below are *measured*, not estimated, on an 11th-gen Intel Core i5-1135G7 (mobile, containerized).

9.1 Circuit

Constraint counts depend on the circom optimization level, so we report both -01 (non-linear plus linear, circom’s default) and -02 (linear constraints eliminated), with -02 as the primary figure, for the primary settlement circuit and for the earlier circuits discussed below.

Circuit	-02	-01 (n.l. + lin.)	Public / private
settlement_verify_v2 (primary)	1,251	1,266 + 1,524	5 / 8
pix_nullifier_verify (keyed, v1)	537	543 + 710	2 / 5
pix_payment_verify (unkeyed, ablation)	510	516 + 635	2 / 4

The unkeyed circuit `pix_payment_verify` is retained only as an *ablation*: it is the variant §3 declares vulnerable (deanonymization via the unkeyed tag), measured so that its cost can be compared against the keyed `pix_nullifier_verify` and against the primary `settlement_verify_v2`. These are three distinct circuits, not three measurements of the same one, and the construction of this paper uses `settlement_verify_v2` exclusively. The primary circuit has 1,260 wires and 4,229 labels, five Poseidon gadgets (arities 3, 4, 2, 1, 2), and Groth16 proofs of 8 field elements (≈ 256 bytes) over BN254.

9.2 Performance

Prover and off-chain verifier are measured in-process with the snarkjs WASM backend; driving snarkjs through its CLI instead adds ≈ 390 ms of Node.js start-up per run. The figures in the table below are *single warm runs* on the stated hardware, not a median over repeated trials. At these circuit sizes (≈ 10 – 15 ms of verification, ≈ 100 – 150 ms of proving), run-to-run noise is comparable to

the differences between circuits: the ablation circuit (510 constraints) timing *above* the 537-constraint circuit cannot be distinguished from noise without a dispersion figure, which we do not have on the stated hardware. A dispersion measurement (median $\pm$ SD over $N = 30$ warm-up-then-timed runs, via `scripts/bench-prove.cjs`) on a separate, uncontrolled machine produced figures several times the single-run numbers below, which points to the uncontrolled machine rather than to the in-process/CLI distinction, so we do not report them. A same-hardware re-run ($N \geq 30$, median $\pm$ SD) on the stated i5-1135G7 is future work (§11).

Circuit	Prove	Verify (off-chain)	verifyProof gas
<code>settlement_verify_v2</code>	151.2 ms	13.6 ms	243,182
<code>pix_nullifier_verify</code>	110.5 ms	14.0 ms	—
<code>pix_payment_verify</code> (ablation)	130.1 ms	14.7 ms	—

We do not quote an on-chain figure for the two-signal ablation: its verifier is not part of the artifact set of this paper, and a figure without a transaction a reader can look up does not belong in the table. The expected direction is clear. Under EIP-1108 each public signal adds one `alt_bn128` scalar multiplication and one addition, $\approx 6,150$ gas, so three extra signals should cost $\approx 18,500$ gas on top of the shared four-pairing check. We report no measured delta until a same-template re-measurement (§11) produces one. The dominant cost of either verifier is the four-pairing check ($\approx 181,000$ gas: $45,000 + 34,000 \times 4$ under EIP-1108, for the pairing precompile of EIP-197).

Gas of the complete calls. The isolated verifier understates what a deployment pays. The figures below are entire transactions on the XDC Apothem public testnet (chainId 51), taken from transaction receipts and listed with their hashes in the artifact manifest.

Call	Gas
<code>lock(c0, deadline)</code> : escrow locks on the agreed terms, with a liveness deadline (§4)	74,982
<code>attest(receipt, signature)</code> : ECDSA recover, checks, tag storage	90,913
<code>finalize(proof): complete settlement</code>	353,062
isolated <code>verifyProof</code> call (Groth16, 5 public signals)	243,182
<code>cancel(c0)</code> : after the deadline, Locked $\rightarrow$ Expired	30,259
<code>attestReversal(reversal, signature)</code>	63,529

Figures are those of the reference deployment (receipt carrying `pc`, the `finalize` cross-check, and the `lock/cancel` liveness timeout). The `finalize` figure varies by a few hundred gas from one proof to the next, because calldata prices zero and non-zero bytes differently, so small differences on that row depend on the proof, not on the logic. The oracle stores the receipt binding as a single word (§4), so the cross-check adds one `keccak256` over 64 bytes and one calldata word rather than a storage slot. An *isolated* call to the verifier contract with the same proof and public signals costs 243,182 gas. That figure includes its own 21,000 base transaction cost and its own proof calldata, neither of which recurs when `finalize` calls the verifier internally, so 243,182 is an upper bound on the verifier’s share of the 353,062, not a component that subtracts cleanly to give the remainder. With that caveat, Groth16 verification is still the bulk of the complete transaction’s cost; the rest is the oracle cross-check (the receipt must be attested *and* carry the same (`c0`, `pc`) the escrow and the proof agree on), the domain-anchor check, the tag consumption and the state transition. The isolated figure alone would overstate the verifier’s call-time cost and understate the complete transaction, which is why the complete-call figure is primary. `cancel` is cheap by comparison, a single state write with no cryptographic check, as expected for a path whose purpose is to release a stuck operation at little cost.

9.3 ZK-Circuit Constraint Analysis

We analysed the circuits with Ecne [11], which verifies that an R1CS constraint system is *well-determined*: every signal is uniquely fixed by the inputs, so no signal is *under-constrained*, the dominant ZK-circuit bug class. Non-vacuous results, under the methodology below, cover the primary settlement circuit and the KYC-inclusion circuit. We call this section *constraint analysis* rather than *formal verification* because it establishes the absence of one bug class in the R1CS, not functional correctness, protocol security or soundness of the underlying cryptographic construction (see Scope of the claim, below).

A pitfall: the vacuous verdict. Ecne decides its verdict by comparing how many *target* variables it uniquely determined against the number of targets, and the targets are the *public outputs* of the R1CS. A circuit written in the common circom style `signal input c; ...; c == h.out;`, with public values declared as inputs and pinned by an equality constraint, compiles to an R1CS with `nPubOut = 0`. The target set is then empty and the verdict reduces to $0 = 0$: the tool prints “*has sound constraints (no trusted functions needed)*” without having established anything about the circuit. All of the circuits in this paper are written in that style, and Ecne’s raw verdict on them is vacuous. The criterion is visible in the tool’s source (`target_unique == length(target_variables)`) and in the R1CS header, but the tool does not flag it. A published Ecne verdict for circuits in the input-plus-equality style should therefore be read against `nPubOut`, and reports should quote targets solved over targets alongside the verdict, as the table below does.

Verification under a corrected methodology. Two changes make the analysis meaningful: declare the derived public signals as `signal output`, so that the target set is non-empty, and run the analysis on the *unoptimized* (-00) constraint system, since the -02 optimizer folds the linear structure that Ecne’s symbolic solver relies on. The analysed circuits are separate source files, `settlement_verify_v2_out.circom` and `kyc_inclusion_verify_out.circom`, that keep the constraint semantics of the originals but declare the derived signals as outputs (`<==` in place of `==`) and drop the `public` list; the `_out` variant of the primary circuit compiles to the same 1,251 constraints at -02. Under this methodology the results are non-vacuous:

Circuit	Outputs	Targets	Signals	Verdict
<code>settlement_verify_v2_out</code> (-00)	declared	5/5	4,229/4,229	sound, no trusted fns
<code>kyc_inclusion_verify_out</code> (-00)	declared	1/1	18,230/18,248	sound, no trusted fns
<code>settlement_verify_v2_out</code> (-02)	declared	4/5	1,016/1,260	<i>inconclusive</i>

The KYC circuit’s target (its single declared output) is solved (1/1), so none of the 18 signals Ecne leaves undetermined is a target; this follows directly from the target count. We have not yet identified those 18 signals against the `.sym` file, and list that as future work (§11).

The -02 row shows the tool failing to conclude on an optimized system: “inconclusive” means Ecne’s solver could not determine one target, not that the circuit is unsound. Reporting the target count (4/5, not simply “fail”) is what lets a reader see that the analysis is incomplete rather than negative.

Scope of the claim. Ecne proves well-determinedness. It does *not* prove functional conformance with a specification: a circuit can be well-determined and still compute the wrong predicate. The guarantee is the absence of one important bug class, not correctness. Tools in the Picus [12] line attack the same under-constrainedness question with a different technique and would be a useful cross-check; we have not run them, and list this as future work. The analysis is complemented by

circumspect [13] (a static linter for circom) and a manual audit against a systematic ZK-vulnerability taxonomy. On the public-key side, the ElGamal/BabyJubJub circuits validate every input point (on-curve via `BabyCheck`, plus a small-order rejection $8 \cdot P \neq \mathcal{O}$) to preclude invalid-curve and small-subgroup confinement.

9.4 Reference deployment and reproducibility

Four kinds of evidence appear in this paper, each supporting a different claim. *Cryptographic evidence* (Groth16 verifies, commitments bind; §7) backs the formal properties. *Constraint-analysis evidence* (Ecne well-determinedness; §9.3) backs the absence of under-constrained signals. *Implementation evidence* (the transactions below: correct calls succeed, incorrect ones revert) backs “the code does what §3–§4 describe”. *Deployment evidence* (a live public testnet) backs only “this runs against a real EVM under real gas costs”; a testnet deployment is not security evidence, and we do not present it as such. The construction of this paper (the two-phase circuit, the attestation oracle and the composition that ties them together) is deployed and exercised on the XDC Apothem public testnet (chainId 51). The settlement path was run end to end: the escrow locks on c_0 before any payment exists, the PSP receipt over c_0 is attested, and the Groth16 proof closes the operation. *No amount, E2EID or charge identifier appears in any of those transactions*, which is the confidentiality property central to this design, and a reader can check it directly against the calldata.

The acceptance rules were also exercised negatively, on-chain, in two batches. The first covers receipt validity: a receipt signed by a non-attestor, a receipt with `settled=false`, an expired receipt, and a replay of an already-attested tag were each mined and reverted. The second covers the cross-checks of `finalize` (§3) and the operation state machine. In *case 5a*, the receipt’s c_0 diverges from the one the proof opens, which is what a PSP that recomputed c_0 over the wrong settled amount would produce. In *case 5b*, the receipt’s `partyCommitment` diverges from the one the proof opens; without this check `partyCommitment` would be a decorative public signal, since a prover could attach any counterparty commitment to a valid proof. In *case 6*, the proof is generated under a domain key k' different from the registered domain anchor K ; the tag-uniqueness argument of §3 holds only if the proof is bound to the registered domain. *Case 7* exercises the reversal path: a short-window instance (600s, deployed only to make this exercisable in minutes) is settled, then reversed within the window via `attestReversal`; `isReleasable` immediately reports `false`, and 660s later, past the window that would otherwise have made an unreversed operation irrevocable, `isIrrevocable` on that tag is still `false`. A reversal blocks release permanently, not merely until the window closes. *Case 8* exercises the liveness timeout (§4): `cancel` on a still-Locked operation reverts before its `lock` deadline and succeeds once the deadline passes (moving it to `Expired`), and a valid proof submitted to `finalize` afterward reverts on-chain, so `cancel` and `finalize` are exercised as the mutually exclusive pair §4 describes. All nine cases were submitted against the artifacts of Assumption (S1) with explicit gas, so that they were recorded on the chain rather than stopped by client-side simulation, each leaving a hash a reader can look up. The four acceptance-rule cases (non-attestor signer, `settled = false`, expired receipt, replayed tag) were sent to the happy-path oracle itself, so the replayed tag is the one that settled the reference operation.

Contract addresses, every transaction hash, the public signals of the settled operation, and the exact commands that reproduce each number in this section are listed in `docs/eprint/ARTIFACTS.md` of the reference repository, <https://github.com/harpo-zk/proof-of-instant-settlement>, at commit `0db97bf91ddc5be475e3d4175a62e241eb80df79`. Reproducibility claims in this section are claims about that exact commit.

A reading note for those transactions: in the reference deployment the deployer, the registered attestor and the registry owner are the same key (`0xbE3c...eE60`), a prototype convenience, not a

design property. The negative cases must be read accordingly: the non-attestor case is signed by a fixed, publicly known throwaway key that is not a registered attestor, and a production deployment separates the three roles (multisig owner, §8).

10 Related Work

Proving fiat payment to release on-chain value. Proving that a fiat payment happened in order to release on-chain value is not new. ZKP2P [6] has done so in production since 2023, using zk-email and zkTLS over consumer payment rails, and is the closest prior art for the general mechanism; it does not target the institutional setting and has no confidential-compliance model. Canton [7] is the incumbent for institutional delivery-versus-payment with privacy, on a permissioned substrate; Brazil’s Drex pilots occupy the same regulated space with a central-bank-operated ledger. Our position is the intersection those leave open: a public EVM chain, an institutional (PSP-attested) rail binding, and compliance answered as predicates.

Confidential transfer on EVM. Rayls/Enygma [4] (Parfin) is the closest peer in the confidential-transfer family: same regulator context, same Groth16/Poseidon/BabyJubJub primitives, central-bank pilots. It provides confidential, EVM-compatible institutional settlement and delivery-versus-payment primitives in its own right, with a *global* auditor view key, which the table below contrasts with our predicate-based, scoped auditor model (§5, §6). Zeto [5] and Nightfall provide generic confidential transfers on EVM. To the best of our knowledge, none of these binds a real instant-payment rail in the sense evaluated here: they answer “was the on-chain transfer well-formed”, not “did the fiat settle”. Our contribution is orthogonal: binding an external instant-payment rail event to confidential EVM state, composable with what these systems do downstream of that point (see Positioning below). For post-quantum auditability, Qurrency [10] adds ML-KEM-based auditor access to Zeto on a Groth16/circom stack; we reuse its open-source ML-KEM circom implementation (§6) and bind it to our rail-anchored commitment. Our compliance predicates (§5) differ from Zeto’s KYC/audit variants in the disclosure model: where those let an authority decrypt the value, ours disclose only a boolean.

	ZKP2P	Rayls	Zeto	Qurrency	This work
Fiat-rail binding	yes	no	no	no	yes
Institutional / regulated rail	no	yes	n/a	n/a	yes
Public EVM chain	yes	partly	yes	yes	yes
Confidential transfer	no	yes	yes	yes	yes
Auditor model	n/a	global key	decrypt	decrypt (PQ)	predicate / scoped
Compliance w/o disclosure	no	no	no	no	yes
Post-quantum audit channel	no	no	no	yes	preliminary
Production maturity	yes	pilots	yes	yes	prototype

Table: feature comparison. “decrypt” means the auditor recovers the plaintext; “predicate” means the auditor learns only a yes/no. On *maturity* the comparison inverts: ZKP2P is in production and Zeto/Qurrency are backed by LF Decentralized Trust; this work is a research prototype. The edge claimed here is the design and the rail binding, not deployment maturity.

Positioning. The table is useful for placement, but the claim is architectural rather than feature-by-feature. *The novelty here is not confidential transfer itself*: Zeto and Rayls/Enygma already do that well, and Paladin’s roadmap addresses atomic composition between private on-chain components.

It is the cryptographic binding layer between an external, real-world instant-payment event and confidential EVM state. Zeto and Paladin answer “is this confidential on-chain state well-formed”; this construction answers “does a specific real payment, attested by a regulated PSP, correspond to this confidential on-chain state”, and hands the resulting commitment forward, unopened, to whatever confidential-asset system consumes it. The natural relationship to Zeto and Paladin is therefore composition, not competition: this construction sits upstream of them, converting an external fiat event into a confidential commitment they could consume as an input. We have not built or measured that composition (§11); the claim is architectural positioning, not a demonstrated integration.

11 Open Problems and Future Work

1. **Trustless attestation** (§8): a synchronous central-bank signature (the heavy ask); optimistic attestation with retrospective rail anchoring (path iv), including its cheap policy form, a periodically signed accumulator of settled E2EIDs, and its engineering form, in-circuit or in-TEE verification of the rail confirmations the PSPs already hold (`pacs.002/pacs.004`); dual attestation via open banking; or zkTLS over the PSP endpoint. Each removes, or converts into ex-post accountability, a different part of the trust. Of these, the retrospective anchor is the only one deployable in principle against artifacts the rail signs today, and it composes with the state machine of §4.2 as an orthogonal flag rather than a change to the settlement path.
2. **Dispute path beyond timeout:** `cancel` (§4) closes the liveness gap with a deadline-based escape, which is sufficient but coarse: it does not let a counterparty contest a still-live operation before the deadline, or distinguish “PSP is slow” from “PSP is refusing”. A richer off-chain dispute path (e.g. an appeals window an aggrieved party can trigger before the deadline) is future work on top of the timeout primitive.
3. **Domain-wide spent-tag registry:** `tagUsed` and the oracle’s attestation record are per deployed instance, so Theorem 3 is instance-scoped; a registry of spent ν indexed by K , consulted by every `Settle` instance of a domain, would make it domain-scoped at the cost of one shared storage write per settlement.
4. **Attestor privacy:** hiding *which* PSP attested, via a ring or threshold signature over the registered attestor set.
5. **Atomic DvP, and a real composition with a confidential-asset system:** the two-phase commitment makes the asset leg lockable on c_0 , but composing it with the payment leg into a single atomic transition remains to be built and measured. The strongest version is an actual integration, even at testnet scale, with a system like Zeto or Paladin consuming this construction’s commitment as an input to a private asset transfer, which would turn the architectural positioning of §10 into a demonstrated one.
6. **Production trusted setup:** a real multi-party ceremony.
7. **Primitive agility:** migration to Poseidon2 and to a universal-setup proving system (PLONK/Halo2).
8. **Full post-quantum binding** (§6): the ceremony for the 1.62M-constraint ML-KEM binding circuit on production hardware.

9. **Verification depth** (§9.3): cross-check with Picus [12], extend the analysis to the optimized constraint system, identify the 18 undetermined non-target signals of `kyc_inclusion_verify` against its `.sym` file, and move from well-determinedness towards functional conformance.
10. **Same-environment benchmark re-run** (§9): a dispersion measurement (median $\pm$ SD, $N \geq 30$) of proving/verification time on the stated i5-1135G7, and a same-template gas re-measurement of the two-signal and five-signal verifiers generated from the same snarkjs codegen, to report a measured per-signal delta.
11. **The attested counterparty commitment as a standalone primitive**: a regulated institution attesting $pc = \text{Poseidon}(\text{id}, \text{salt}_p)$ over the identifier it processed provides a reusable identity anchor, since third-party ZK predicates (credit, licensing, jurisdiction) can consume the same pc without any change to this protocol. Generalizing it beyond the three predicates shipped here is future work.

Availability

The settlement circuit, its snarkjs-generated Groth16 verifier, the attestation-oracle contract, the compliance-predicate circuits, and the end-to-end flow scripts are available in the reference repository, <https://github.com/harpo-zk/proof-of-instant-settlement>, at commit `0db97bf91ddc5be475e3d4175a62e241eb80df79`, together with the artifact manifest described in §9. This is a research prototype validated on a public testnet; it has no production deployment, and the claims of this paper are limited to what the listed transactions and artifacts demonstrate.

Acknowledgments

The authors used large language model assistants (Anthropic Claude) during this work, to draft and edit parts of the text, to review the circuits and contracts, and to cross-check figures and references against the artifacts. All content, including every reference and every reported number, was reviewed and verified by the authors, who take full responsibility for it.

References

- [1] J. Groth. *On the Size of Pairing-Based Non-interactive Arguments*. EUROCRYPT 2016, LNCS 9666, pp. 305–326. Springer, 2016. IACR ePrint 2016/260.
- [2] K. Bagheri, M. Kohlweiss, J. Siim, M. Volkhov. *Another Look at Extraction and Randomization of Groth’s zk-SNARK*. IACR ePrint 2020/811.
- [3] L. Grassi, D. Khovratovich, C. Rechberger, A. Roy, M. Schofnegger. *Poseidon: A New Hash Function for Zero-Knowledge Proof Systems*. USENIX Security 2021.
- [4] M. Yaksetig, J. Xu. *Rayls: A Novel Design for CBDCs*. IACR ePrint 2025/1639, 2025. See also M. Yaksetig, P. M. F. Pereira, S. Yang, M. Nejadgholi, J. Xu. *Rayls II: Fast, Private, and Compliant CBDCs*. IACR ePrint 2025/1638, 2025 (the Enygma protocol, revised).
- [5] LF Decentralized Trust, Paladin project. *Zeto: Privacy-preserving implementations of fungible and non-fungible tokens, using UTXO as the underlying transaction model*. <https://github.com/LFDT-Paladin/zeto>, accessed September 2026.

- [6] ZKP2P (now Peer). *Peer Documentation*. <https://docs.peer.xyz>, accessed September 2026.
- [7] Digital Asset. *Canton Network: A Network of Networks for Smart Contract Applications*. White paper, updated January 2024. <https://www.digitalasset.com/hubfs/Canton/Canton%20Network%20-%20White%20Paper.pdf>.
- [8] F. Zhang, D. Maram, H. Malvai, S. Goldfeder, A. Juels. *DECO: Liberating Web Data Using Decentralized Oracles for TLS*. ACM CCS 2020.
- [9] National Institute of Standards and Technology. *FIPS 203: Module-Lattice-Based Key-Encapsulation Mechanism Standard*. August 2024. <https://csrc.nist.gov/pubs/fips/203/final>.
- [10] A. R. Choudhuri, S. Garg, M. Gregoire, K. Lee, M. Lodder, H. Montgomery, G. V. Policharla, J. Zhang. *Qurrency: a quantum-secure, private, and auditable platform for digital assets*. IACR ePrint 2026/015, 2026. Implementation: <https://github.com/LFDT-Paladin/zeto>.
- [11] F. Wang. *Ecne: an engine for verifying the soundness of R1CS constraints*. 0xPARC, 2022. <https://github.com/franklynwang/EcneProject>.
- [12] S. Pailoor, Y. Chen, F. Wang, C. Rodríguez, J. Van Geffen, J. Morton, M. Chu, B. Gu, Y. Feng, I. Dillig. *Automated Detection of Under-Constrained Circuits in Zero-Knowledge Proofs*. PLDI 2023. IACR ePrint 2023/512.
- [13] Trail of Bits. *Circomspect: a static analyzer and linter for the Circom zero-knowledge DSL*. 2022. <https://github.com/trailofbits/circomspect>.
- [14] Banco Central do Brasil. *Catálogo de Serviços do SFN, Volume VI — Pagamentos Instantâneos*, versão 5.12. https://www.bcb.gov.br/content/estabilidadefinanceira/cedsfn/Catalogos/Catalogo_de_Servicos_do_SFN_Volume_VI_Versao_512.pdf.
- [15] Receita Federal do Brasil. *Instrução Normativa RFB nº 2.229, de 15 de outubro de 2024* (CNPJ alfanumérico, implantação progressiva a partir de 31/07/2026). <https://www.gov.br/receita-federal/pt-br/aceso-a-informacao/acoes-e-programas/programas-e-atividades/cnpj-alfanumerico>.

A Pix instantiation of CanonicalID

This appendix gives $\text{CanonicalID}_{\text{Pix}}$, the concrete canonicalization rule this prototype and its pilot deployment use for the Pix rail (§2). It is a deployment profile, not part of the core protocol: any other instant-payment rail plugs in its own $\text{CanonicalID}_{\text{rail}}$ in its place, unchanged elsewhere.

CPF/CNPJ. The bare ASCII characters of the identifier with all punctuation stripped and no padding: 11 decimal digits for a CPF, and 14 characters for a CNPJ, upper-cased, exactly as extracted from the debtor field of the payment message. Since Instrução Normativa RFB nº 2.229/2024 [15], rolled out from July 31, 2026, the first 12 of those 14 characters may be digits or uppercase letters (existing all-numeric CNPJs remain valid unchanged); the final two check digits are computed by the Receita Federal’s modulo-11 algorithm over the ASCII value of each character and are always numeric. Hence $\text{CanonicalID}_{\text{Pix}}(\text{CNPJ})$ needs no branching for the two formats, only the fixed 14-character, upper-cased, punctuation-stripped rule above.

E2EID / txid. The exact bytes returned by the PSP API: UTF-8, no case-folding, no trimming, no re-serialization (§2). This is the rule stated in §2 for these two fields, repeated here so that the appendix is self-contained as the Pix deployment profile.